

register. However, if we use the approach of the previous section to generate code for a block, there is a good chance that after x has been computed in a block it will remain in a register if there are subsequent uses of x in that block. Thus we count a savings of one for each use of x in loop L that is not preceded by an assignment to x in the same block. We also save two units if we can avoid a store of x at the end of a block. Thus if x is allocated a register, count a savings of two for each block of L for which x is live on exit and in which x is assigned a value.

On the debit side, if x is live on entry to the loop header, we must load x into its register just before entering loop L . This load costs two units. Similarly, for each exit block B of loop L at which x is live on entry to some successor of B outside of L , we must store x at a cost of two. However, on the assumption that the loop is iterated many times, we may neglect these debits since they occur only once each time we enter the loop. Thus an approximate formula for the benefit to be realized from allocating a register to x within loop L is:

$$\sum_{\text{blocks } B \text{ in } L} (\text{use}(x, B) + 2 * \text{live}(x, B)) \quad (9.4)$$

where $\text{use}(x, B)$ is the number of times x is used in B prior to any definition of x ; $\text{live}(x, B)$ is 1 if x is live on exit from B and is assigned a value in B , and $\text{live}(x, B)$ is 0 otherwise. Note that (9.4) is approximate, because not all blocks in a loop are executed with equal frequency and also because (9.4) was based on the assumption that a loop is iterated "many" times. On other machines a formula analogous to (9.4), but possibly quite different from it, would have to be developed.

Example 9.6. Consider the basic blocks in the inner loop depicted in Fig. 9.13, where jump and conditional jump statements have been omitted. Assume registers R0, R1, and R2 are allocated to hold values throughout the loop. Variables live on entry into and on exit from each block are shown in Fig. 9.13 for convenience, immediately above and below each block, respectively. There are some subtle points about live variables that we address in Chapter 10. For example, notice that both e and f are live at the end of B_1 , but of these, only e is live on entry to B_2 and only f on entry to B_3 . In general, the variables live at the end of a block are the union of those live at the beginning of each of its successor blocks.

To evaluate (9.4) for $x = a$ we observe that a is live on exit from B_1 and is assigned a value there, but is not live on exit from B_2 , B_3 , or B_4 . Thus, $\sum_{B \text{ in } L} 2 * \text{live}(a, B) = 2$. Also, $\text{use}(a, B_1) = 0$, since a is defined in B_1 before any use. Also, $\text{use}(a, B_2) = \text{use}(a, B_3) = 1$ and $\text{use}(a, B_4) = 0$. Thus, $\sum_{B \text{ in } L} \text{use}(a, B) = 2$. Hence the value of (9.4) for $x = a$ is 4. That is, four units of cost can be saved by selecting a for one of the global registers. The values of (9.4) for b , c , d , e , and f are 6, 3, 6, 4, and 4, respectively.

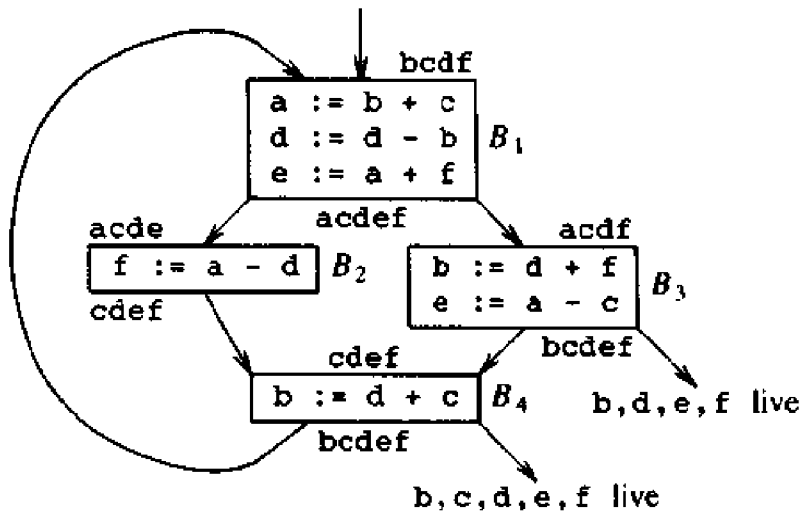


Fig. 9.13. Flow graph of an inner loop.

Thus we may select a , b , and d , for registers R_0 , R_1 , and R_2 , respectively. Using R_0 for e or f instead of a would be another choice with the same apparent benefit. Fig. 9.14 shows the assembly code generated from Fig. 9.13, assuming that the strategy of Section 9.6 is used to generate code for each block. We do not show the generated code for the omitted conditional or unconditional jumps that end each block in Fig. 9.13, and we therefore do not show the generated code as a single stream, as it would appear in practice. It is worth noting that, if we did not adhere strictly to our strategy of reserving R_0 , R_1 , and R_2 , we could use

```

SUB  R2, R0
MOV  R0, f
  
```

for B_2 , saving a unit since a is not live on exit from B_2 . A similar saving could be realized at B_3 . $\square$

Register Assignment for Outer Loops

Having assigned registers and generated code for inner loops, we may apply the same idea to progressively larger loops. If an outer loop L_1 contains an inner loop L_2 , the names allocated registers in L_2 need not be allocated registers in $L_1 - L_2$. However, if name x is allocated a register in loop L_1 but not L_2 , we must store x on entrance to L_2 and load x if we leave L_2 and enter a block of $L_1 - L_2$. Similarly, if we choose to allocate x a register in L_2 but not L_1 we must load x on entrance to L_2 and store x on exit from L_2 . We leave as an exercise the derivation of a criterion for selecting names to be allocated registers in an outer loop L , given that choices have already been made for all loops nested within L .

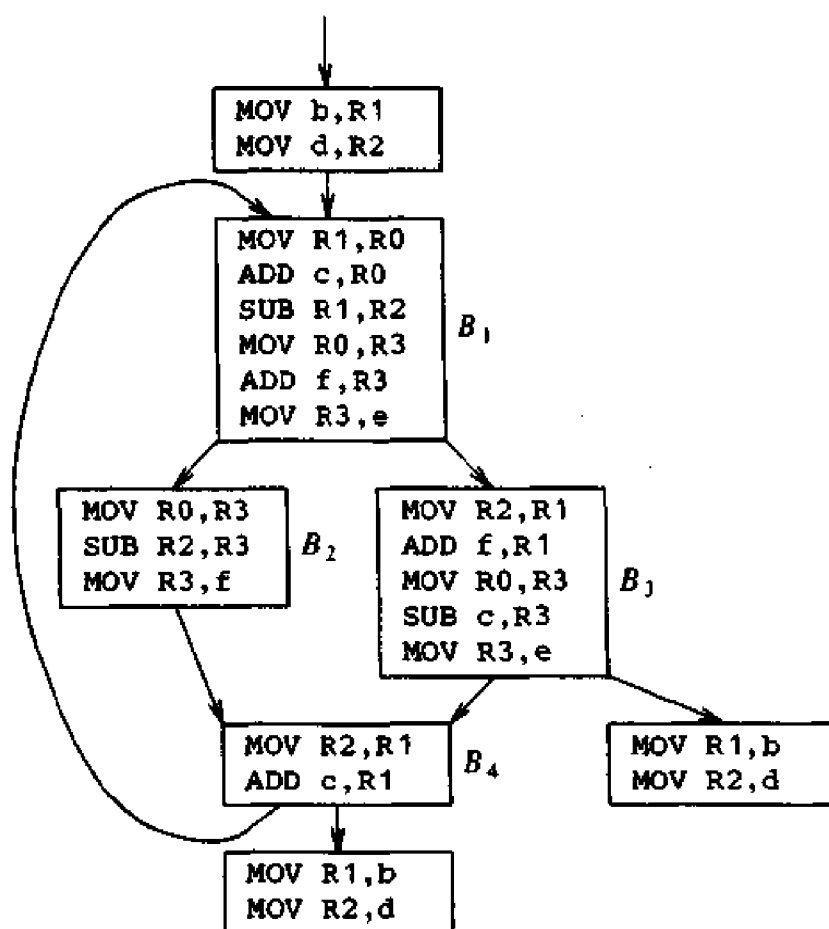


Fig. 9.14. Code sequence using global register assignment.

Register Allocation by Graph Coloring

When a register is needed for a computation but all available registers are in use, the contents of one of the used registers must be stored (*spilled*) into a memory location in order to free up a register. Graph coloring is a simple, systematic technique for allocating registers and managing register spills.

In this method, two passes are used. In the first, target-machine instructions are selected as though there were an infinite number of symbolic registers; in effect, names used in the intermediate code become names of registers and the three-address statements become machine-language statements. If access to variables requires instructions that use stack pointers, display pointers, base registers, or other quantities that assist access, then we assume that these quantities are held in registers reserved for each purpose. Normally, their use is directly translatable into an access mode for an address mentioned in a machine instruction. If access is more complex, the access must be broken into several machine instructions, and a temporary symbolic register (or several) may need to be created.

Once the instructions have been selected, a second pass assigns physical registers to symbolic ones. The goal is to find an assignment that minimizes the cost of the spills.

In the second pass, for each procedure a *register-interference graph* is constructed in which the nodes are symbolic registers and an edge connects two nodes if one is live at a point where the other is defined. For example, a register-interference graph for Figure 9.13 would have nodes for names *a* and *d*. In block B_1 , *a* is live at the second statement, which defines *d*; therefore, in the graph there would be an edge between the nodes for *a* and *d*.

An attempt is made to color the register-interference graph using k colors, where k is the number of assignable registers. (A graph is said to be *colored* if each node has been assigned a color in such a way that no two adjacent nodes have the same color.) A color represents a register, and the coloring makes sure that no two symbolic registers that can interfere with each other are assigned the same physical register.

Although the problem of determining whether a graph is k -colorable is NP-complete in general, the following heuristic technique can usually be used to do the coloring quickly in practice. Suppose a node n in a graph G has fewer than k neighbors (nodes connected to n by an edge). Remove n and its edges from G to obtain a graph G' . A k -coloring of G' can be extended to a k -coloring of G by assigning n a color not assigned to any of its neighbors.

By repeatedly eliminating nodes having fewer than k edges from the register-interference graph, either we obtain the empty graph, in which case we can produce a k -coloring for the original graph by coloring the nodes in the reverse order in which they were removed, or we obtain a graph in which each node has k or more adjacent nodes. In the latter case a k -coloring is no longer possible. At this point a node is spilled by introducing code to store and reload the register. Then the interference graph is appropriately modified and the coloring process resumed. Chaitin [1982] and Chaitin et al. [1981] describe several heuristics for choosing the node to spill. A general rule is to avoid introducing spill code into inner loops.

9.8 THE DAG REPRESENTATION OF BASIC BLOCKS

Directed acyclic graphs (dags) are useful data structures for implementing transformations on basic blocks. A dag gives a picture of how the value computed by each statement in a basic block is used in subsequent statements of the block. Constructing a dag from three-address statements is a good way of determining common subexpressions (expressions computed more than once) within a block, determining which names are used inside the block but evaluated outside the block, and determining which statements of the block could have their computed value used outside the block.

A *dag for a basic block* (or just *dag*) is a directed acyclic graph with the following labels on nodes:

1. Leaves are labeled by unique identifiers, either variable names or

constants. From the operator applied to a name we determine whether the *l*-value or *r*-value of a name is needed; most leaves represent *r*-values. The leaves represent initial values of names, and we subscript them with 0 to avoid confusion with labels denoting "current" values of names as in (3) below.

2. Interior nodes are labeled by an operator symbol.
3. Nodes are also optionally given a sequence of identifiers for labels. The intention is that interior nodes represent computed values, and the identifiers labeling a node are deemed to have that value.

It is important not to confuse dags with flow graphs. Each node of a flow graph can be represented by a dag, since each node of the flow graph stands for a basic block.

```

(1)  t1 := 4 * i
(2)  t2 := a [ t1 ]
(3)  t3 := 4 * i
(4)  t4 := b [ t3 ]
(5)  t5 := t2 * t4
(6)  t6 := prod + t5
(7)  prod := t6
(8)  t7 := i + 1
(9)  i := t7
(10) if i <= 20 goto (1)

```

Fig. 9.15. Three-address code for block B_2 .

Example 9.7. Figure 9.15 shows the three-address code corresponding to block B_2 of Fig. 9.9. Statement numbers starting from (1) have been used for convenience. The corresponding dag is shown in Fig. 9.16. We discuss the significance of the dag after giving an algorithm to construct it. For the time being let us observe that each node of the dag represents a formula in terms of the leaves, that is, the values possessed by variables and constants upon entering the block. For example, the node labeled t_4 in Fig. 9.16 represents the formula

$$b[4 * i]$$

that is, the value of the word whose address is $4*i$ bytes offset from address b , which is the intended value of t_4 . □

Dag Construction

To construct a dag for a basic block, we process each statement of the block in turn. When we see a statement of the form $x := y + z$, we look for the

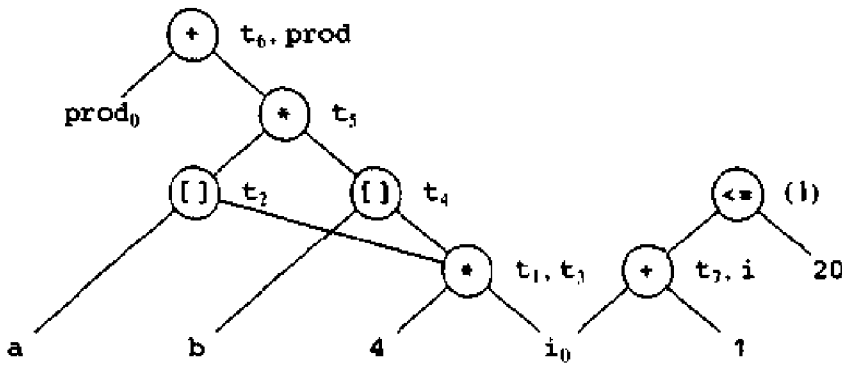


Fig. 9.16. Dag for block of Fig. 9.15.

nodes that represent the “current” values of y and z . These could be leaves, or they could be interior nodes of the dag if y and/or z had been evaluated by previous statements of the block. We then create a node labeled $+$ and give it two children; the left child is the node for y , the right the node for z . Then we label this node x . However, if there is already a node denoting the same value as $y + z$, we do not add the new node to the dag, but rather give the existing node the additional label x .

Two details should be mentioned. First, if x (not x_0) had previously labeled some other node, we remove that label, since the “current” value of x is the node just created. Second, for an assignment such as $x := y$ we do not create a new node. Rather, we append label x to the list of names on the node for the “current” value of y .

We now give the algorithm to compute a dag from a block. The algorithm is almost the same as Algorithm 5.1, except for the additional list of identifiers we attach to each node here. The reader should be warned that this algorithm may not operate correctly if there are assignments to arrays, if there are indirect assignments through pointers, or if one memory location can be referred to by two or more names, due to EQUIVALENCE statements or the correspondences between actual and formal parameters of a procedure call. We discuss the modifications necessary to handle these situations at the end of this section.

Algorithm 9.2. Constructing a dag.

Input. A basic block.

Output. A dag for the basic block containing the following information:

1. A *label* for each node. For leaves the label is an identifier (constants permitted), and for interior nodes, an operator symbol.
2. For each node a (possibly empty) list of attached identifiers (constants not permitted here).

Method. We assume the appropriate data structures are available to create nodes with one or two children, with a distinction between “left” and “right” children in the latter case. Also available in the structure is a place for a label for each node and the facility to create a linked list of attached identifiers for each node.

In addition to these components, we need to maintain the set of all identifiers (including constants) for which there is a node associated. The node could be either a leaf labeled by that identifier or an interior node with that identifier on its attached identifier list. We assume the existence of a function $node(identifier)$, which, as we build the dag, returns the most recently created node associated with $identifier$. Intuitively, $node(identifier)$ is the node of the dag that represents the value that $identifier$ has at the current point in the dag construction process. In practice, an entry in the symbol-table record for $identifier$ would indicate the value of $node(identifier)$.

The dag construction process is to do the following steps (1) through (3) for each statement of the block, in turn. Initially, we assume there are no nodes, and $node$ is undefined for all arguments. Suppose the “current” three-address statement is either (i) $x := y \text{ op } z$, (ii) $x := \text{op } y$, or (iii) $x := y$.⁷ We refer to these as cases (i), (ii), and (iii). We treat a relational operator like $\text{if } i \leq 20 \text{ goto}$ as case (i), with x undefined.

1. If $node(y)$ is undefined, create a leaf labeled y , and let $node(y)$ be this node. In case (i), if $node(z)$ is undefined, create a leaf labeled z and let that leaf be $node(z)$.
2. In case (i), determine if there is a node labeled op , whose left child is $node(y)$ and whose right child is $node(z)$. (This check is to catch common subexpressions.) If not, create such a node. In either event, let n be the node found or created. In case (ii), determine whether there is a node labeled op , whose lone child is $node(y)$. If not, create such a node, and let n be the node found or created. In case (iii), let n be $node(y)$.
3. Delete x from the list of attached identifiers for $node(x)$. Append x to the list of attached identifiers for the node n found in (2) and set $node(x)$ to n . □

Example 9.8. Let us return to the block of Fig. 9.15 and see how the dag of Fig. 9.16 is constructed for it. The first statement is $t_1 := 4 * i$. In step (1), we must create leaves labeled 4 and i_0 . (We use the subscript 0, as before, to help distinguish labels from attached identifiers in pictures, but the subscript is not really part of the label.) In step (2), we create a node labeled $*$, and in step (3) we attach identifier t_1 to it. Figure 9.17(a) shows the dag at this stage.

⁷ Operators are assumed to have at most two arguments. The generalization to three or more arguments is straightforward.

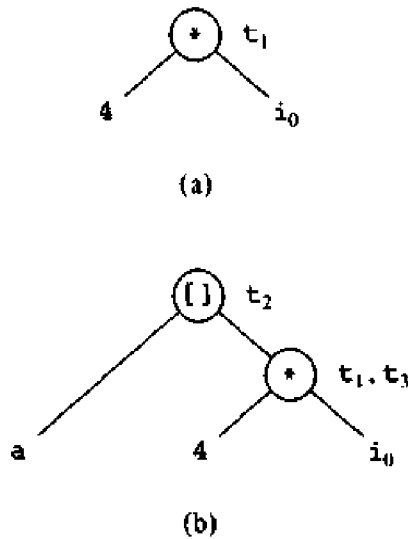


Fig. 9.17. Steps in the dag construction process.

For the second statement, $t_2 := a[t_1]$, we create a new leaf labeled a and find the previously created $node(t_1)$. We also create a new node labeled $[]$ to which we attach the nodes for a and t_1 as children.

For statement (3), $t_3 := 4 * i$, we determine that $node(4)$ and $node(i)$ already exist. Since the operator is $*$, we do not create a new node for statement (3), but rather append t_3 on the identifier list for node t_1 . The resulting dag is shown in Fig. 9.17(b). The value-number method of Section 5.2 can be used to discover quickly that the node for $4 * i$ already exists.

We invite the reader to complete the construction of the dag. We mention only the steps taken for statement (9), $i := t_7$. Before statement (9), $node(i)$ is the leaf labeled i_0 . Statement (9) is an instance of case (iii); therefore, we find $node(t_7)$, append i to its identifier list, and set $node(i)$ to $node(t_7)$. This is one of only two statements — the other is statement (7) — where the value of $node$ changes for an identifier. It is this change that ensures that the new node for i is the left child of the $\leq$ operator node constructed for statement (10). $\square$

Applications of Dags

There are several pieces of useful information that we can obtain as we are executing Algorithm 9.2. First, note that we automatically detect common subexpressions. Second, we can determine which identifiers have their values used in the block; they are exactly those for which a leaf is created in step (1) at some time. Third, we can determine which statements compute values that could be used outside the block. They are exactly those statements S whose node n constructed or found in step (2) still has $node(x) = n$ at the end of the dag construction, where x is the identifier assigned by statement S . (Equivalently, x is still an attached identifier for n .)

Example 9.9. In Example 9.8, all statements meet the above constraint because the only times *node* is redefined — for *prod* and *i* — the previous value of *node* was a leaf. Thus all interior nodes can have their value used outside the block. Now, suppose we inserted before statement (9) a new statement *s* that assigned a value to *i*. At statement *s* we would create a node *m* and set *node*(*i*) = *m*. However, at statement (9) we would redefine *node*(*i*). Thus the value computed at statement *s* could not be used outside the block. $\square$

Another important use to which the dag may be put is to reconstruct a simplified list of quadruples taking advantage of common subexpressions and not performing assignments of the form $x := y$ unless absolutely necessary. That is, whenever a node has more than one identifier on its attached list, we check which, if any, of those identifiers are needed outside the block. As we mentioned, finding live variables for the end of a block requires a data-flow analysis called “live-variable analysis” discussed in Chapter 10. However, in many cases we may assume no temporary name such as $t_1, t_2, \dots, t_7$ in Fig. 9.15 is needed outside the block. (But beware of how logical expressions are translated; one expression may wind up spread over several basic blocks.)

We may in general evaluate the interior nodes of the dag in any order that is a topological sort of the dag. In a topological sort, a node is not evaluated until all of its children that are interior nodes have been evaluated. As we evaluate a node, we assign its value to one of its attached identifiers *x*, preferring one whose value is needed outside the block. We may not, however, choose *x* if there is another node *m* whose value was also held by *x* such that *m* has been evaluated and is still “live.” Here, we define *m* to be live if its value is needed outside the block or if *m* has a parent which has not yet been evaluated.

If there are additional attached identifiers $y_1, y_2, \dots, y_k$ for a node *n* whose values are also needed outside the block, we assign to them with statements $y_1 := x, y_2 := x, \dots, y_k := x$. If *n* has no attached identifiers at all (this could happen if, say, *n* was created by an assignment to *x*, but *x* was subsequently reassigned), we create a new temporary name to hold the value of *n*. The reader should beware that in the presence of pointer or array assignments, not every topological sort of a dag is permissible; we shall attend to this matter shortly.

Example 9.10. Let us reconstruct a basic block from the dag of Fig. 9.16, ordering nodes in the same order as they were created: $t_1, t_2, t_4, t_5, t_6, t_7, (1)$. Note that statements (3) and (7) of the original block did not create new nodes, but added labels t_3 and *prod* to the identifier lists of nodes t_1 and t_6 , respectively. We assume none of the temporaries t_i are needed outside the block.

We begin with the node representing $4*i$. This node has two identifiers attached, t_1 and t_3 . Let us pick t_1 to hold the value $4*i$, so the first reconstructed statement is

$$t_1 := 4 * i$$

just as in the original basic block. The second node considered is labeled t_2 . The statement constructed from this node is

$$t_2 := a[t_1]$$

also as before. The node considered next is labeled t_4 from which we generate the statement

$$t_4 := b[t_1]$$

The latter statement uses t_1 as an argument rather than t_3 as in the original basic block, because t_1 is the name chosen to carry the value $4*i$.

Next we consider the node labeled t_5 and generate the statement

$$t_5 := t_2 * t_4$$

For the node labeled t_6 , *prod*, we select *prod* to carry the value, since that identifier and not t_6 will (presumably) be needed outside the block. Like t_3 , temporary t_6 disappears. The next statement generated is

$$\text{prod} := \text{prod} + t_5$$

Similarly, we choose *i* rather than t_7 to carry the value $i+1$. The last two statements generated are

$$\begin{aligned} i &:= i + 1 \\ \text{if } i \leq 20 &\text{ goto } (1) \end{aligned}$$

Note that the ten statements of Fig. 9.15 have been reduced to seven by taking advantage of the common subexpressions exposed during the dag construction process, and by eliminating unnecessary assignments. $\square$

Arrays, Pointers, and Procedure Calls

Consider the basic block:

$$\begin{aligned} x &:= a[i] \\ a[j] &:= y \\ z &:= a[i] \end{aligned} \tag{9.5}$$

If we use Algorithm 9.2 to construct the dag for (9.5), $a[i]$ would become a common subexpression, and the "optimized" block would turn out to be

$$\begin{aligned} x &:= a[i] \\ z &:= x \\ a[j] &:= y \end{aligned} \tag{9.6}$$

However, (9.5) and (9.6) compute different values for *z* in the case $i = j$ and $y \neq a[i]$. The problem is that when we assign to an array *a*, we may be changing the *r*-value of expression $a[i]$, even though *a* and *i* do not change. It is therefore necessary that when processing an assignment to array

a , we *kill* all nodes labeled $[]$, whose left argument is a plus or minus a constant (possibly zero).⁸ That is, we make these nodes ineligible to be given an additional identifier label, preventing them from being falsely recognized as common subexpressions. It is thus required that we keep a bit for each node telling whether or not it has been killed. Further, for each array a mentioned in the block, it is convenient to have a list of all nodes currently not killed but that must be killed should we assign to an element of a .

A similar problem occurs if we have an assignment such as $*p := w$, where p is a pointer. If we do not know what p might point to, every node currently in the dag being built must be killed in the sense above. If node n labeled a is killed and there is a subsequent assignment to a , we must create a new leaf for a and use that leaf rather than n . We later consider the constraints on the order of evaluation caused by the killing of nodes.

In Chapter 10, we discuss methods whereby we could discover that p could only point to some subset of the identifiers. If p could only point to r or s , then only $node(r)$ and $node(s)$ must be killed. It is also conceivable that we could discover that $i = j$ is impossible in block (9.5), in which case the node for $a[i]$ need not be killed by $a[j] := y$. However, the latter type of discovery is not usually worth the trouble.

A procedure call in a basic block kills all nodes, since in the absence of knowledge about the called procedure, we must assume that any variable may be changed as a side effect. Chapter 10 discusses how we may establish that certain identifiers are not changed by a procedure call, and then nodes for these identifiers need not be killed.

If we intend to reassemble the dag into a basic block and may not want to use the order in which the nodes of the dag were created, then we must indicate in the dag that certain apparently independent nodes must be evaluated in a certain order. For example, in (9.5), the statement $z := a[i]$ must follow $a[j] := y$, which must follow $x := a[i]$. Let us introduce certain edges $n \rightarrow m$ in the dag that do not indicate that m is an argument of n , but rather that evaluation of n must follow evaluation of m in any computation of the dag. The rules to be enforced are the following.

1. Any evaluation of or assignment to an element of array a must follow the previous assignment to an element of that array if there is one.
2. Any assignment to an element of array a must follow any previous evaluation of a .
3. Any use of any identifier must follow the previous procedure call or indirect assignment through a pointer if there is one.
4. Any procedure call or indirect assignment through a pointer must follow all previous evaluations of any identifier.

⁸ Note that the argument of $[]$ indicating the array name could be a itself, or an expression like $a-4$. In the latter case, the node a would be a grandchild, rather than a child of the node $[]$.

That is, when reordering code, uses of an array *a* may not cross each other, and no statement may cross a procedure call or an assignment through a pointer.

9.9 PEEPHOLE OPTIMIZATION

A statement-by-statement code-generation strategy often produces target code that contains redundant instructions and suboptimal constructs. The quality of such target code can be improved by applying “optimizing” transformations to the target program. The term “optimizing” is somewhat misleading because there is no guarantee that the resulting code is optimal under any mathematical measure. Nevertheless, many simple transformations can significantly improve the running time or space requirement of the target program, so it is important to know what kinds of transformations are useful in practice.

A simple but effective technique for locally improving the target code is *peephole optimization*, a method for trying to improve the performance of the target program by examining a short sequence of target instructions (called the *peephole*) and replacing these instructions by a shorter or faster sequence, whenever possible. Although we discuss peephole optimization as a technique for improving the quality of the target code, the technique can also be applied directly after intermediate code generation to improve the intermediate representation.

The peephole is a small, moving window on the target program. The code in the peephole need not be contiguous, although some implementations do require this. It is characteristic of peephole optimization that each improvement may spawn opportunities for additional improvements. In general, repeated passes over the target code are necessary to get the maximum benefit. In this section, we shall give the following examples of program transformations that are characteristic of peephole optimizations:

- redundant-instruction elimination
- flow-of-control optimizations
- algebraic simplifications
- use of machine idioms

Redundant Loads and Stores

If we see the instruction sequence

```
(1) MOV    R0, a
(2) MOV    a, R0
```

(9.7)

we can delete instruction (2) because whenever (2) is executed, (1) will ensure that the value of *a* is already in register *R0*. Note that if (2) had a label⁹ we

⁹ One advantage of generating assembly code is that labels will be present, facilitating peephole optimizations such as this. If machine code is generated, and peephole optimization is desired, we can use a bit to mark the instructions that would have labels.

could not be sure that (1) was always executed immediately before (2) and so we could not remove (2). Put another way, (1) and (2) have to be in the same basic block for this transformation to be safe.

While target code such as (9.7) would not be generated if the algorithm suggested in Section 9.6 were used, it might be if a more naive algorithm like the one mentioned at the beginning of Section 9.1 were used.

Unreachable Code

Another opportunity for peephole optimization is the removal of unreachable instructions. An unlabeled instruction immediately following an unconditional jump may be removed. This operation can be repeated to eliminate a sequence of instructions. For example, for debugging purposes, a large program may have within it certain segments that are executed only if a variable `debug` is 1. In C, the source code might look like:

```
#define debug 0
...
if ( debug ) {
    print debugging information
}
```

In the intermediate representation the if-statement may be translated as:

```
    if debug = 1 goto L1
    goto L2                                (9.8)
L1: print debugging information
L2:
```

One obvious peephole optimization is to eliminate jumps over jumps. Thus, no matter what the value of `debug`, (9.8) can be replaced by:

```
    if debug ≠ 1 goto L2
    print debugging information            (9.9)
L2:
```

Now, since `debug` is set to 0 at the beginning of the program,¹⁰ constant propagation should replace (9.9) by

```
    if 0 ≠ 1 goto L2
    print debugging information            (9.10)
L2:
```

As the argument of the first statement of (9.10) evaluates to a constant **true**, it can be replaced by `goto L2`. Then all the statements that print debugging aids are manifestly unreachable and can be eliminated one at a time.

¹⁰ To tell that `debug` has the value 0 we need to do a global "reaching definitions" data-flow analysis, as discussed in Chapter 10.

Flow-of-Control Optimizations

The intermediate code generation algorithms in Chapter 8 frequently produce jumps to jumps, jumps to conditional jumps, or conditional jumps to jumps. These unnecessary jumps can be eliminated in either the intermediate code or the target code by the following types of peephole optimizations. We can replace the jump sequence

```
        goto L1
        ...
L1: goto L2
```

by the sequence

```
        goto L2
        ...
L1: goto L2
```

If there are now no jumps to L1,¹¹ then it may be possible to eliminate the statement L1: goto L2 provided it is preceded by an unconditional jump. Similarly, the sequence

```
        if a < b goto L1
        ...
L1: goto L2
```

can be replaced by

```
        if a < b goto L2
        ...
L1: goto L2
```

Finally, suppose there is only one jump to L1 and L1 is preceded by an unconditional goto. Then the sequence

```
        goto L1
        ...
L1: if a < b goto L2
L3:
```

(9.11)

may be replaced by

```
        if a < b goto L2
        goto L3
        ...
L3:
```

(9.12)

While the number of instructions in (9.11) and (9.12) is the same, we

¹¹ If this peephole optimization is attempted, we can count the number of jumps to each label in the symbol-table entry for that label; a search of the code is not necessary.

sometimes skip the unconditional jump in (9.12), but never in (9.11). Thus, (9.12) is superior to (9.11) in execution time.

Algebraic Simplification

There is no end to the amount of algebraic simplification that can be attempted through peephole optimization. However, only a few algebraic identities occur frequently enough that it is worth considering implementing them. For example, statements such as

$$x := x + 0$$

or

$$x := x * 1$$

are often produced by straightforward intermediate code-generation algorithms, and they can be eliminated easily through peephole optimization.

Reduction in Strength

Reduction in strength replaces expensive operations by equivalent cheaper ones on the target machine. Certain machine instructions are considerably cheaper than others and can often be used as special cases of more expensive operators. For example, x^2 is invariably cheaper to implement as $x*x$ than as a call to an exponentiation routine. Fixed-point multiplication or division by a power of two is cheaper to implement as a shift. Floating-point division by a constant can be implemented (approximated) as multiplication by a constant, which may be cheaper.

Use of Machine Idioms

The target machine may have hardware instructions to implement certain specific operations efficiently. Detecting situations that permit the use of these instructions can reduce execution time significantly. For example, some machines have auto-increment and auto-decrement addressing modes. These add or subtract one from an operand before or after using its value. The use of these modes greatly improves the quality of code when pushing or popping a stack, as in parameter passing. These modes can also be used in code for statements like $i:=i+1$.

9.10 GENERATING CODE FROM DAGS

In this section, we show how to generate code for a basic block from its dag representation. The advantage of doing so is that from a dag we can more easily see how to rearrange the order of the final computation sequence than we can starting from a linear sequence of three-address statements or quadruples. Central to our discussion is the case where the dag is a tree. For this case we can generate code that we can prove is optimal under such criteria as

program length or the fewest number of temporaries used. This algorithm for optimal code generation from a tree is also useful when the intermediate code is a parse tree.

Rearranging the Order

Let us briefly consider how the order in which computations are done can affect the cost of resulting object code. Consider the following basic block whose dag representation is shown in Fig. 9.18 (the dag happens to be a tree).

```

t1 := a + b
t2 := c + d
t3 := e - t2
t4 := t1 - t3

```

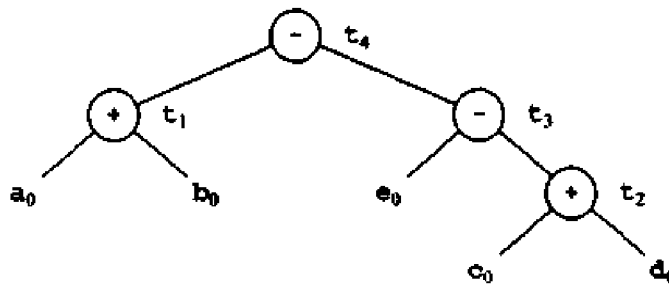


Fig. 9.18. Dag for basic block.

Note that the order is the one we would naturally obtain from a syntax-directed translation of the expression $(a+b) - (e - (c+d))$ by the algorithm of Section 8.3.

If we generate code for the three-address statements using the algorithm in Section 9.6 we get the code sequence of Fig. 9.19 (assuming two registers R0 and R1 are available, and only t_4 is live on exit).

```

MOV  a, R0
ADD  b, R0
MOV  c, R1
ADD  d, R1
MOV  R0, t1
MOV  e, R0
SUB  R1, R0
MOV  t1, R1
SUB  R0, R1
MOV  R1, t4

```

Fig. 9.19. Code sequence.

On the other hand suppose we rearranged the order of the statements so that the computation of t_1 occurs immediately before that of t_4 as:

```

 $t_2 := c + d$ 
 $t_3 := e - t_2$ 
 $t_1 := a + b$ 
 $t_4 := t_1 - t_3$ 

```

Then, using the code-generation algorithm of Section 9.6, we get the code sequence of Fig. 9.20. (Again, only R0 and R1 are available.) By performing the computation in this order, we have been able to save two instructions, MOV R0, t_1 (which stores the value of R0 in memory location t_1) and MOV t_1 , R1 (which reloads the value of t_1 in register R1).

```

MOV  c, R0
ADD  d, R0
MOV  e, R1
SUB  R0, R1
MOV  a, R0
ADD  b, R0
SUB  R1, R0
MOV  R0,  $t_4$ 

```

Fig. 9.20. Revised code sequence.

A Heuristic Ordering for Dags

The reason the above reordering improved the code was that the computation of t_4 was made to follow immediately after the computation of t_1 , its left operand in the tree. That this arrangement is beneficial should be clear. The left argument for the computation of t_4 must be in a register for efficient computation of t_4 , and computing t_1 immediately before t_4 ensures that will be the case.

In selecting an ordering for the nodes of a dag, we are only constrained to be sure that the order preserves the edge relationships of the dag. Recall from Section 9.8 that those edges can represent either the operator-operand relationship or implied constraints due to possible interactions among procedure calls, array assignments, or pointer assignments. We propose the following heuristic ordering algorithm, which attempts as far as possible to make the evaluation of a node immediately follow the evaluation of its leftmost argument. The algorithm of Fig. 9.21 produces the ordering in reverse.

Example 9.11. The algorithm of Fig. 9.21 applied to the tree of Fig. 9.18 yields the order from which the code of Fig. 9.20 was produced. For a more complete example, consider the dag of Fig. 9.22.

```

(1)  while unlisted interior nodes remain do begin
(2)      select an unlisted node  $n$ , all of whose parents have
          been listed;
(3)      list  $n$ ;
(4)      while the leftmost child  $m$  of  $n$  has no unlisted parents
          and is not a leaf do
          /* since  $n$  was just listed,  $m$  is not yet listed */
          begin
(5)          list  $m$ ;
(6)           $n := m$ 
          end
      end
end

```

Fig. 9.21. Node listing algorithm.

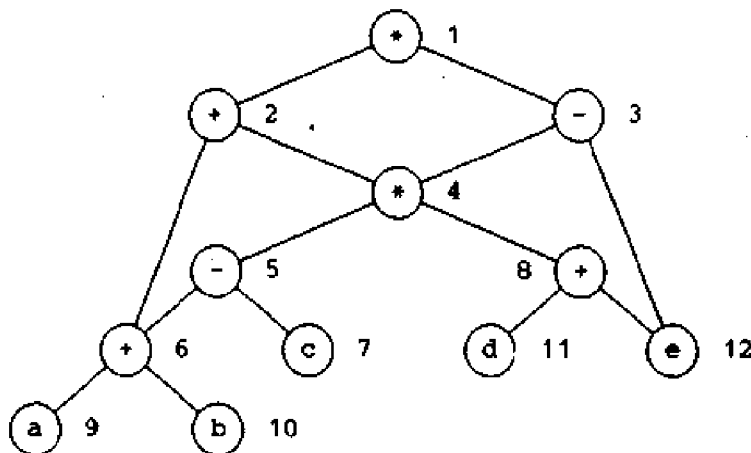


Fig. 9.22. A dag.

Initially the only node with no unlisted parents is 1, so we set $n=1$ at line (2) and list 1 at line (3). Now the left argument of 1, which is 2, has its parents listed, so we list 2 and set $n=2$ at line (6). Now at line (4) we find the leftmost child of 2, which is 6, has an unlisted parent, 5. Thus we select a new n at line (2), and node 3 is the only candidate. We list 3 and then proceed down its left chain, listing 4, 5, and 6. This leaves only 8 among the interior nodes, so we list that. The resulting list is 1234568 so the suggested order of evaluation is 8654321. This ordering corresponds to the sequence of three-address statements:

```

 $t_8 := d + e$ 
 $t_6 := a + b$ 
 $t_5 := t_6 - c$ 

```

```

t4 := t5 * t8
t3 := t4 - e
t2 := t6 + t4
t1 := t2 * t3

```

which will yield optimal code for the dag on our machine whatever the number of registers, if the code-generation algorithm of Section 9.6 is used. It should be noted that in this example our ordering heuristic never had any choices to make at step (2), but in general it may have many choices. $\square$

Optimal Ordering for Trees

It turns out that for the machine model of Section 9.2 we can give a simple algorithm to determine the optimal order in which to evaluate statements in a basic block when the dag representation of the block is a tree. Optimal order here means the order that yields the shortest instruction sequence, over all instruction sequences that evaluate the tree. This algorithm, modified to take into account register pairs and other target-machine vagaries, has been used in compilers for Algol, Bliss, and C.

The algorithm has two parts. The first part labels each node of the tree, bottom-up, with an integer that denotes the fewest number of registers required to evaluate the tree with no stores of intermediate results. The second part of the algorithm is a tree traversal whose order is governed by the computed node labels. The output code is generated during the tree traversal.

Intuitively, the algorithm works, given the two operands of a binary operator, by first evaluating the operand requiring more registers (the harder operand). If the register requirements of both operands are the same, either operand can be evaluated first.

The Labeling Algorithm

We use the term "left leaf" to mean a node that is a leaf and the leftmost descendant of its parent. All other leaves are referred to as "right leaves."

The labeling can be done by visiting nodes in a bottom-up order so that a node is not visited until all its children are labeled. The order in which parse tree nodes are created is suitable if the parse tree is used as intermediate code, so in this case, the labels can be computed as a syntax-directed translation. Figure 9.23 gives the algorithm for computing the label at node n . In the important special case that n is a binary node and its children have labels l_1 and l_2 , the formula of line (6) reduces to

$$label(n) = \begin{cases} \max(l_1, l_2) & \text{if } l_1 \neq l_2 \\ l_1 + 1 & \text{if } l_1 = l_2 \end{cases}$$

Example 9.12. Consider the tree in Fig. 9.18. A postorder traversal¹² of the

¹² A postorder traversal recursively visits the subtrees rooted at children $n_1, n_2, \dots, n_k$ of a node n , then n . It is the order in which nodes of a parse tree are created in a bottom-up parse.

- (1) **if** n is a leaf **then**
- (2) **if** n is the leftmost child of its parent **then**
- (3) $label(n) := 1$
- (4) **else** $label(n) := 0$
- else begin** /* n is an interior node */
- (5) let $n_1, n_2, \dots, n_k$ be the children of n ordered by *label*,
 so $label(n_1) \geq label(n_2) \geq \dots \geq label(n_k)$;
- (6) $label(n) := \max_{1 \leq i \leq k} (label(n_i) + i - 1)$
- end**

Fig. 9.23. Label computation.

nodes visits the nodes in the order $a \ b \ t_1 \ e \ c \ d \ t_2 \ t_3 \ t_4$. Postorder is always an appropriate order in which to do the label computations. Node a is labeled 1 since it is a left leaf. Node b is labeled 0 since it is a right leaf. Node t_1 is labeled 1 because the labels of its children are unequal and the maximum label of a child is 1. Figure 9.24 shows the labeled tree that results. It implies that two registers are needed to evaluate t_4 and, in fact, two registers are needed just to evaluate t_3 . $\square$

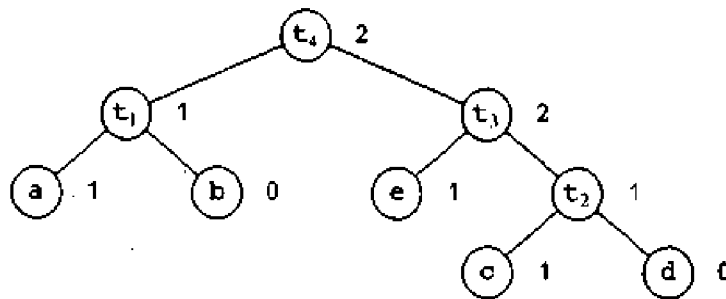


Fig. 9.24. Labeled tree.

Code Generation from a Labeled Tree

We now present the algorithm that takes as input a labeled tree T and produces as output a machine code sequence that evaluates T into $R0$. ($R0$ can then be stored into the appropriate memory location.) We assume T has only binary operators. The generalization to operators with an arbitrary number of operands is not hard, and it is left as an exercise.

The algorithm uses the recursive procedure $gencode(n)$ to produce machine code evaluating the subtree of T with root n into a register. The procedure $gencode$ uses a stack $rstack$ to allocate registers. Initially, $rstack$ contains all available registers, which we assume are $R0, R1, \dots, R(r-1)$, in this

order. A call of *gencode* may find a subset of the registers, perhaps in a different order, on *rstack*. When *gencode* returns, it leaves the registers on *rstack* in the same order it found them. The resulting code computes the value of the tree *T* in the top register on *rstack*.

The function *swap(rstack)* interchanges the top two registers on *rstack*. The use of *swap* is to make sure that a left child and its parent are evaluated into the same register.

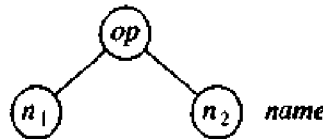
The procedure *gencode* uses a stack *tstack* to allocate temporary memory locations. We assume *tstack* initially contains *T0*, *T1*, *T2*, In practice, *tstack* need not be implemented as a list, if we just keep track of that *i* such that *Ti* is currently on top. The contents of *tstack* is always a suffix of *T0*, *T1*,

The statement $X := \text{pop}(\text{stack})$ means "pop *stack* and assign the value popped to *X*." Conversely, we use *push(stack, X)* to mean "push *X* onto *stack*;" *top(stack)* refers to the value on top of *stack*.

The code-generation algorithm is to call *gencode* on the root of *T*, where *gencode* is the procedure shown in Fig. 9.25. It can be explained by examining each of the five cases. For case 0, we have a subtree of the form



That is, *n* is a leaf and the leftmost child of its parent. Therefore we generate just a load instruction. In case 1, we have a subtree of the form



for which we generate code to evaluate *n1* into register $R = \text{top}(\text{rstack})$ followed by the instruction *op name, R*. In case 2, we have a subtree of the form



where *n1* can be evaluated without stores but *n2* is harder to evaluate (i.e., requires more registers) than *n1*. For this case, we swap the top two registers on *rstack*, then evaluate *n2* into $R = \text{top}(\text{rstack})$. We remove *R* from *rstack* and evaluate *n1* into $S = \text{top}(\text{rstack})$. Note that *S* was the register initially on top of *rstack* at the beginning of case 2. We then generate the instruction *op R, S*, which produces the value of *n* (the node labeled *op*) in register *S*. Another call to *swap* leaves *rstack* as it was when this call of *gencode* began.

```

procedure gencode(n);
begin
  /* case 0 */
  if n is a left leaf representing operand name and n is
    the leftmost child of its parent then
    print 'MOV' || name || ',' || top(rstack)
  else if n is an interior node with operator op, left child n1,
    and right child n2 then
    /* case 1 */
    if label(n2) = 0 then begin
      let name be the operand represented by n2;
      gencode(n1);
      print op || name || ',' || top(rstack)
    end
    /* case 2 */
    else if  $1 \leq \text{label}(n_1) < \text{label}(n_2)$  and  $\text{label}(n_1) < r$  then begin
      swap(rstack);
      gencode(n2);
      R := pop(rstack); /* n2 was evaluated into register R */
      gencode(n1);
      print op || R || ',' || top(rstack);
      push(rstack, R);
      swap(rstack)
    end
    /* case 3 */
    else if  $1 \leq \text{label}(n_2) \leq \text{label}(n_1)$  and  $\text{label}(n_2) < r$  then begin
      gencode(n1);
      R := pop(rstack); /* n1 was evaluated into register R */
      gencode(n2);
      print op || top(rstack) || ',' || R;
      push(rstack, R)
    end
    /* case 4, both labels  $\geq r$ , the total number of registers */
    else begin
      gencode(n2);
      T := pop(tstack);
      print 'MOV' || top(rstack) || ',' || T;
      gencode(n1);
      push(tstack, T);
      print op || T || ',' || top(rstack)
    end
  end
end

```

Fig. 9.25. The function *gencode*.

Case 3 is similar to case 2 except that here the left subtree is harder and is evaluated first. There is no need to swap registers here.

Case 4 occurs when both subtrees require r or more registers to evaluate without stores. Since we must use a temporary memory location, we first evaluate the right subtree into the temporary T , then the left subtree, and finally the root.

Example 9.13. Let us generate code for the labeled tree in Fig. 9.24 with $rstack = R_0, R_1$ initially. The sequence of calls to *gencode* and code printing steps is shown in Fig. 9.26. Shown alongside in brackets is the contents of *rstack* at the time of each call, with the top at the right end. The code sequence here is a permutation of that in Fig. 9.20. $\square$

```

gencode( $t_4$ )      [ $R_1 R_0$ ]          /* case 2 */
  gencode( $t_3$ )    [ $R_0 R_1$ ]          /* case 3 */
    gencode( $e$ )    [ $R_0 R_1$ ]          /* case 0 */
      print MOV  $e, R_1$ 
    gencode( $t_2$ )  [ $R_0$ ]              /* case 1 */
      gencode( $c$ )  [ $R_0$ ]              /* case 0 */
        print MOV  $c, R_0$ 
      print ADD  $d, R_0$ 
    print SUB  $R_0, R_1$ 
  gencode( $t_1$ )    [ $R_0$ ]              /* case 1 */
    gencode( $a$ )    [ $R_0$ ]              /* case 0 */
      print MOV  $a, R_0$ 
    print ADD  $b, R_0$ 
  print SUB  $R_1, R_0$ 

```

Fig. 9.26. Trace of *gencode* routine.

We can prove that *gencode* produces optimal code on expressions for our machine model, assuming that no algebraic properties of operators are taken into account and assuming there are no common subexpressions. The proof, left as an exercise, is based on showing that any code sequence must perform

1. an operation for each interior node,
2. a load for each leaf which is the leftmost child of its parent, and
3. a store for every node both of whose children have labels equal to or greater than r .

Since *gencode* produces exactly these steps, it is optimal.

Multiregister Operations

We can modify our labeling algorithm to handle operations like multiplication, division, or a function call, which normally require more than one register to perform. Simply modify step (6) of Fig. 9.23, the labeling algorithm, so

$label(n)$ is always at least the number of registers required by the operation. For example, if a function call is assumed to require all r registers, replace line (6) by $label(n) = r$. If multiplication requires two registers, in the binary case use

$$label(n) = \begin{cases} \max(2, l_1, l_2) & \text{if } l_1 \neq l_2 \\ l_1 + 1 & \text{if } l_1 = l_2 \end{cases}$$

where l_1 and l_2 are the labels of the children of n .

Unfortunately, this modification will not guarantee that a register-pair is available for a multiplication or division, or for multiple-precision operations. A useful trick on some machines is to pretend that multiplication and division require three registers. If *swap* is never used in *gencode*, then *rstack* will always contain consecutive high-numbered registers, $i, i+1, \dots, r-1$ for some i . Thus the first three of these are sure to include a register pair. By taking advantage of the fact that many operations are commutative we can often avoid using case 2 of *gencode*, the case that calls *swap*. Also, even if *rstack* does not contain three consecutive registers at the top, we have a very good chance of finding a register-pair somewhere in *rstack*.

Algebraic Properties

If we may assume algebraic laws for various operators, we have the opportunity to replace a given tree T by one with smaller labels (to avoid stores in case 4 of *gencode*) and/or fewer left leaves (to avoid loads in case 0). For example, since $+$ is normally regarded as being commutative, we may replace the tree of Fig. 9.27(a) by that of Fig. 9.27(b), reducing the number of left leaves by one and possibly lowering some labels as well.

Since $+$ is usually treated as being associative as well as being commutative, we may take a cluster of nodes labeled $+$ as in Fig. 9.27(c) and replace it by a left chain as in Fig. 9.27(d). To minimize the label of the root we need only arrange that T_{i_1} is one of T_1, T_2, T_3 , and T_4 with a largest label and that T_{i_1} is not a leaf unless all of $T_1, \dots, T_4$ are.

Common Subexpressions

When there are common subexpressions in a basic block, the corresponding dag will no longer be a tree. The common subexpressions will correspond to nodes with more than one parent, called *shared nodes*. We can no longer apply the labeling algorithm or *gencode* directly. In fact, common subexpressions make code generation markedly more difficult from a mathematical point of view. Bruno and Sethi [1976] showed that optimal code generation for dags on a one-register machine is NP-complete. Aho, Johnson, and Ullman [1977a] showed that even with an unlimited number of registers the problem remains NP-complete. The difficulty arises in trying to determine an optimal order in which to evaluate a dag in the cheapest way.

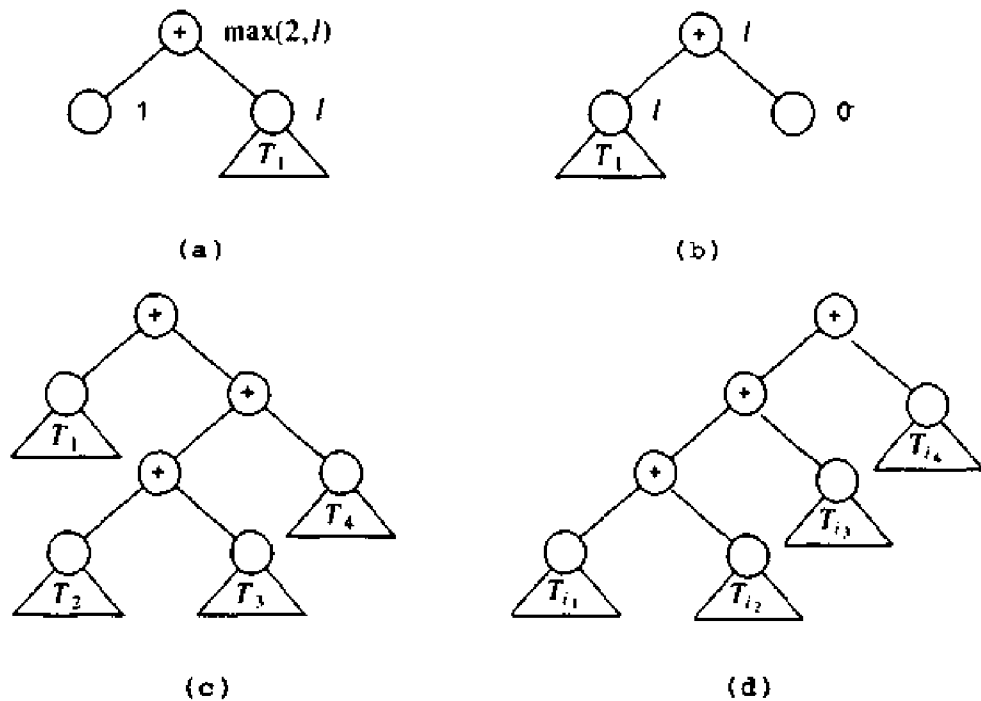


Fig. 9.27. Commutative and associative transformations.

In practice, we can obtain a reasonable solution if we partition the dag into a set of trees by finding for each root and/or shared node n , the maximal subtree with n as root that includes no other shared nodes, except as leaves. For example, the dag of Fig. 9.22 can be partitioned into the trees of Fig. 9.28. Each shared node with p parents appears as a leaf in at most p trees. Nodes with more than one parent in the same tree can be turned into as many leaves as necessary, so no leaf has multiple parents.

Once we have partitioned the dag into trees in this fashion, we can order the evaluation of the trees and use any of the preceding algorithms to generate code for each tree. The order of the trees must be such that shared values that are leaves of a tree must be available when the tree is evaluated. The shared quantities can be computed and stored into memory (or kept in registers if enough registers are available). While this process does not necessarily generate optimal code, it will often be satisfactory.

9.11 DYNAMIC PROGRAMMING CODE-GENERATION ALGORITHM

In the previous section, the procedure *gencode* produces optimal code from an expression tree using an amount of time that is a linear function of the size of the tree. This procedure works for machines in which all computation is done in registers and in which instructions consist of an operator applied to two registers or to a register and a memory location.

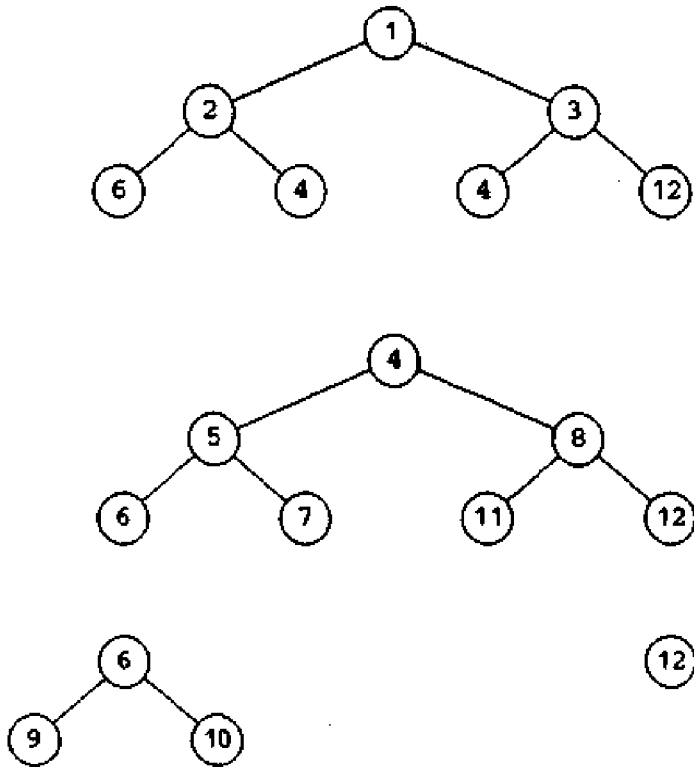


Fig. 9.28. Partition into trees.

An algorithm based on the principle of dynamic programming can be used to extend the class of machines for which optimal code can be generated from expression trees in linear time. The dynamic programming algorithm applies to a broad class of register machines with complex instruction sets.

A Class of Register Machines

The dynamic programming algorithm can be used to generate code for any machine with r interchangeable registers $R_0, R_1, \dots, R_{r-1}$ and instructions of the form $R_i := E$ where E is any expression containing operators, registers, and memory locations. If E involves one or more registers, then R_i must be one of those registers. This machine model includes the machine introduced in Section 9.2.

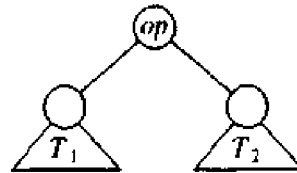
For example, the instruction `ADD R0, R1` would be represented as $R_1 := R_1 + R_0$. The instruction `ADD *R0, R1` would be represented as $R_1 := R_1 + \text{ind } R_0$ where `ind` stands for the indirection operator.

We assume a machine has a load instruction $R_i := M$, a store instruction $M := R_i$, and a register-to-register copy instruction $R_i := R_j$. For simplicity, we also assume every instruction costs one unit, although the dynamic programming algorithm can easily be modified to work even if each instruction has its own cost.

The Principle of Dynamic Programming

The dynamic programming algorithm partitions the problem of generating optimal code for an expression into subproblems of generating optimal code for the subexpressions of the given expression. As a simple example, consider an expression E of the form $E_1 + E_2$. An optimal program for E is formed by combining optimal programs for E_1 and E_2 , in one or the other order, followed by code to evaluate the operator $+$. The subproblems of generating optimal code for E_1 and E_2 are solved similarly.

An optimal program produced by the dynamic programming algorithm has an important property. It evaluates an expression $E = E_1 \text{ op } E_2$ "contiguously." We can appreciate what this means by looking at the syntax tree T for E :



Here T_1 and T_2 are trees for E_1 and E_2 , respectively.

Contiguous Evaluation

We say a program P evaluates a tree T *contiguously* if it first evaluates those subtrees of T that need to be computed into memory. Then, it evaluates the remainder of T either in the order T_1, T_2 , and then the root, or in the order T_2, T_1 , and then the root, in either case using the previously computed values from memory whenever necessary. As an example of noncontiguous evaluation, P might first evaluate part of T_1 leaving the value in a register (instead of memory), next evaluate T_2 , and then return to evaluate the rest of T_1 .

For the register machine defined above, we can prove that given any machine-language program P to evaluate an expression tree T , we can find an equivalent program P' such that

1. P' is of no higher cost than P ,
2. P' uses no more registers than P , and
3. P' evaluates the tree in a contiguous fashion.

This result implies that every expression tree can be evaluated optimally by a contiguous program.

By way of contrast, machines with even-odd register pairs such as the IBM System/370 machines do not always have optimal contiguous evaluations. For these machines, we can give examples of expression trees in which an optimal machine language program must first evaluate into a register a portion of the left subtree of the root, then a portion of the right subtree, then another part of the left subtree, then another part of the right, and so on. This type of

oscillation is unnecessary for an optimal evaluation of any expression tree using the general register machine.

The contiguous evaluation property defined above says that for any expression tree T there always exists an optimal program that consists of optimal programs for subtrees of the root, followed by an instruction to evaluate the root. This property allows us to use the dynamic programming algorithm to generate an optimal program for T .

The Dynamic Programming Algorithm

The dynamic programming algorithm proceeds in three phases. Suppose the target machine has r registers. In the first phase, compute bottom-up for each node n of the expression tree T an array C of costs, in which the i th component $C[i]$ is the optimal cost of computing the subtree S rooted at n into a register, assuming i registers are available for the computation, for $1 \leq i \leq r$. The cost includes whatever loads and stores are necessary to evaluate S in the given number of registers. It also includes the cost of computing the operator at the root of S . The zeroth component of the cost vector is the optimal cost of computing the subtree S into memory. The contiguous evaluation property ensures that an optimal program for S can be generated by considering combinations of optimal programs only for the subtrees of the root of S . This restriction reduces the number of cases that need to be considered.

To compute $C[i]$ at node n , consider each machine instruction $R := E$ whose expression E matches the subexpression rooted at node n . By examining the cost vectors at the corresponding descendants of n , determine the costs of evaluating the operands of E . For those operands of E that are registers, consider all possible orders in which the corresponding subtrees of T can be evaluated into registers. In each ordering, the first subtree corresponding to a register operand can be evaluated using i available registers, the second using $i - 1$ registers, and so on. To account for node n , add in the cost of the instruction $R := E$ that was used to match node n . The value $C[i]$ is then the minimum cost over all possible orders.

The cost vectors for the entire tree T can be computed bottom up in time linearly proportional to the number of nodes in T . It is convenient to store at each node the instruction used to achieve the best cost for $C[i]$ for each value of i . The smallest cost in the vector for the root of T gives the minimum cost of evaluating T .

In the second phase of the algorithm, traverse T , using the cost vectors to determine which subtrees of T must be computed into memory. In the third phase, traverse each tree using the cost vectors and associated instructions to generate the final target code. The code for the subtrees computed into memory locations is generated first. These two phases can also be implemented to run in time linearly proportional to the size of the expression tree.

Example 9.14. Consider a machine having two registers $R0$ and $R1$, and the following instructions, each of unit cost:

$R_i := M_j$
 $R_i := R_i \text{ op } R_j$
 $R_i := R_i \text{ op } M_j$
 $R_i := R_j$
 $M_i := R_i$

In these instructions, R_i is either R_0 or R_1 , and M_j is a memory location.

Let us apply the dynamic programming algorithm to generate optimal code for the syntax tree in Fig. 9.29. In the first phase, we compute the cost vectors shown at each node. To illustrate this cost computation, consider the cost vector at the leaf a . $C[0]$, the cost of computing a into memory, is 0 since it is already there. $C[1]$, the cost of computing a into a register, is 1 since we can load it into a register with the instruction $R_0 := a$. $C[2]$, the cost of loading a into a register with two registers available, is the same as that with one register available. The cost vector at leaf a is therefore $(0, 1, 1)$.

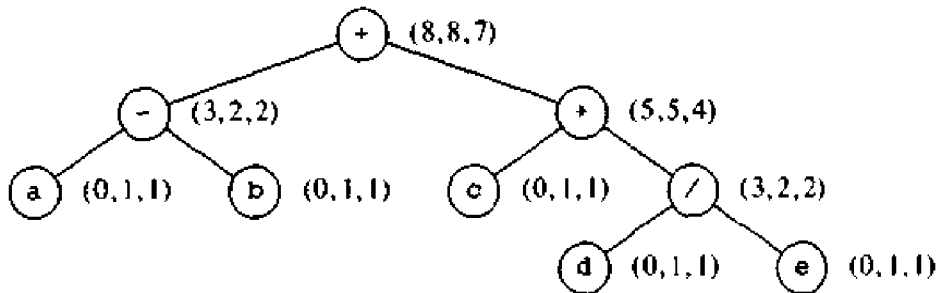


Fig. 9.29. Syntax tree for $(a-b) + c*(d/e)$ with cost vector at each node.

Consider the cost vector at the root. We first determine the minimum cost of computing the root with one and two registers available. The machine instruction $R_0 := R_0 + M$ matches the root, because the root is labeled with the operator $+$. Using this instruction, the minimum cost of evaluating the root with one register available is the minimum cost of computing its right subtree into memory, plus the minimum cost of computing its left subtree into the register, plus 1 for the instruction. No other way exists. The cost vectors at the right and left children of the root show that the minimum cost of computing the root with one register available is $5+2+1 = 8$.

Now consider the minimum cost of evaluating the root with two registers available. Three cases arise depending on which instruction is used to compute the root and in what order the left and right subtrees of the root are evaluated.

1. Compute the left subtree with two registers available into register R_0 , compute the right subtree with one register available into register R_1 , and use the instruction $R_0 := R_0 + R_1$ to compute the root. This sequence has cost $2+5+1 = 8$.

2. Compute the right subtree with two registers available into R1, compute the left subtree with one register available into R0, and use the instruction $R0 := R0 + R1$. This sequence has cost $4+2+1 = 7$.
3. Compute the right subtree into memory location M, compute the left subtree with two registers available into register R0, and use the instruction $R0 := R0 + M$. This sequence has cost $5+2+1 = 8$.

The second choice gives the minimum cost 7.

The minimum cost of computing the root into memory is determined by adding one to the minimum cost of computing the root with all registers available; that is, we compute the root into a register and then store the result. The cost vector at the root is therefore (8,8,7).

From the cost vectors we can easily construct the code sequence by making a traversal of the tree. From the tree in Fig. 9.29, assuming two registers are available, an optimal code sequence is

```

R0 := c
R1 := d
R1 := R1 / e
R0 := R0 * R1
R1 := a
R1 := R1 - b
R1 := R1 + R0

```

□

This technique, originally developed in Aho and Johnson [1976], has been used in a number of compilers, including the second version of S. C. Johnson's portable C compiler, PCC2. The technique facilitates retargeting because of the applicability of the dynamic programming technique to a broad class of machines.

9.12 CODE-GENERATOR GENERATORS

Code generation involves picking an evaluation order for operations, assigning registers to hold values, and selecting the appropriate target-language instructions to implement the operators in the intermediate representation. Even if we assume the order of evaluation is given and registers are allocated by a separate mechanism, the problem of deciding what instructions to use can be a large combinatorial task, especially on a machine rich in addressing modes. In this section, we present tree-rewriting techniques that can be used to construct the instruction selection phase of a code generator automatically from a high-level specification of the target machine.

Code Generation by Tree Rewriting

Throughout this section, the input to the code-generation process will be a sequence of trees at the semantic level of the target machine. The trees are what we might get after inserting the run-time addresses into the intermediate

representation, as described in Section 9.3.

Example 9.15. Figure 9.30 contains a tree for the assignment statement $a[i] := b + 1$ in which a and i are locals whose run-time addresses are given as offsets const_a and const_i from SP , the register containing the pointer to the beginning of the current activation record. The array a is stored on the run-time stack. The assignment to $a[i]$ is an indirect assignment in which the r -value of the location for $a[i]$ is set to the r -value of the expression $b + 1$. The address of array a is given by adding the value of the constant const_a to the contents of register SP ; the value of i is in the location obtained by adding the value of the constant const_i to the contents of register SP . The variable b is a global in memory location mem_b . For simplicity, we assume all variables are of type character.

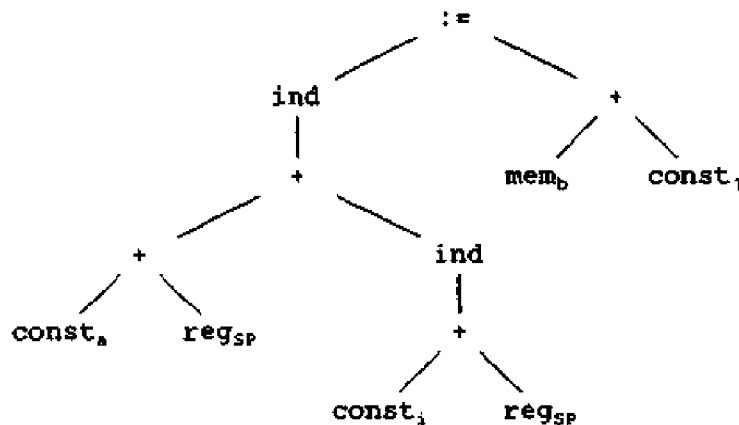


Fig. 9.30. Intermediate-code tree for $a[i] := b + 1$.

In the tree, the `ind` operator treats its argument as a memory address. As the left child of an assignment operator, the `ind` node gives the location into which the r -value on the right side of the assignment operator is to be stored. If an argument of a `+` or `ind` operator is a memory location or a register, then the contents of that memory location or register are taken as the value. The leaves in the tree are type attributes with subscripts; the subscript indicates the value of the attribute. $\square$

The target code is generated during a process in which the input tree is reduced into a single node by applying a sequence of tree-rewriting rules to the tree. Each tree-rewriting rule is a statement of the form

$$\text{replacement} \leftarrow \text{template} \{ \text{action} \}$$

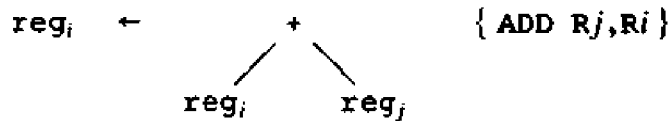
where

1. *replacement* is a single node,
2. *template* is a tree, and
3. *action* is a code fragment, as in a syntax-directed translation scheme.

A set of tree-rewriting rules is called a *tree-translation scheme*.

Each tree template represents a computation performed by the sequence of machine instructions emitted by the associated action. Usually, a template corresponds to a single machine instruction. The leaves of the template are attributes with subscripts, as in the input tree. Often, certain restrictions apply to the values of the subscripts in the templates; these restrictions are specified as semantic predicates that must be satisfied before the template is said to match. For example, a predicate might specify that the value of a constant fall in a certain range.

A tree-translation scheme is a convenient way to represent the instruction-selection phase of a code generator. As an example of a tree-rewriting rule, consider the rule for the register-to-register add instruction:



This rule is used as follows. If the input tree contains a subtree that matches this tree template, that is, a subtree whose root is labeled by the operator $+$ and whose left and right children are quantities in registers i and j , then we can replace that subtree by a single node labeled reg_i and emit the instruction $\text{ADD } R_j, R_i$ as output. More than one template may match a subtree at a given time; we shall describe shortly some mechanisms for deciding which rule to apply in cases of conflict. We assume register allocation is done before code selection.

Example 9.16. Figure 9.31 contains tree-rewriting rules for a few instructions of our target machine. These rules will be used in a running example throughout this section. The first two rules correspond to load instructions, the next two to store instructions, and the remainder to indexed loads and additions. Note that rule (8) requires the value of the constant to be one. This condition would be specified by a semantic predicate. $\square$

A tree-translation scheme works in the following way. Given an input tree, the templates in the tree-rewriting rules are applied to its subtrees. If a template matches, the matching subtree in the input tree is replaced with the replacement node of the rule and the action associated with the rule is done. If the action contains a sequence of machine instructions, the instructions are emitted. This process is repeated until the tree is reduced to a single node, or until no more templates match. The sequence of machine instructions generated as the input tree is reduced to a single node constitutes the output of the tree-translation scheme on the given input tree.

The process of specifying a code generator becomes similar to that of using a syntax-directed translation scheme to specify a translator. We write a tree-translation scheme to describe the instruction set of a target machine. In practice, we would like to find a scheme that causes a minimal-cost instruction

(1)	$\text{reg}_i \leftarrow \text{const}_c$	{ MOV #c, Ri }
(2)	$\text{reg}_i \leftarrow \text{mem}_a$	{ MOV a, Ri }
(3)	$\text{mem} \leftarrow$ <pre> * / \ mem_a reg_i </pre>	{ MOV Ri, a }
(4)	$\text{mem} \leftarrow$ <pre> * / \ ind reg_j reg_i </pre>	{ MOV Rj, *Ri }
(5)	$\text{reg}_i \leftarrow$ <pre> ind + / \ const_c reg_j </pre>	{ MOV c(Rj), Ri }
(6)	$\text{reg}_i \leftarrow$ <pre> + / \ reg_i ind + / \ const_c reg_j </pre>	{ ADD c(Rj), Ri }
(7)	$\text{reg}_i \leftarrow$ <pre> + / \ reg_i reg_j </pre>	{ ADD Rj, Ri }
(8)	$\text{reg}_i \leftarrow$ <pre> + / \ reg_i const_1 </pre>	{ INC Ri }

Fig. 9.31. Tree-rewriting rules for some target-machine instructions.

sequence to be generated for each input tree. Several tools are available to help build a code generator automatically from a tree-translation scheme.

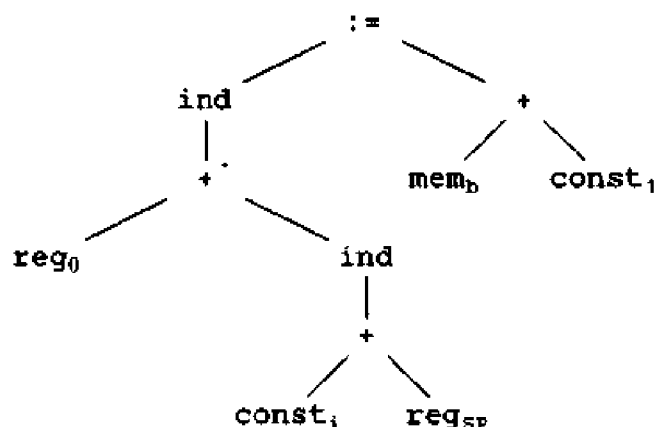
Example 9.17. Let us use the tree-translation scheme in Fig. 9.31 to generate code for the input tree in Fig. 9.30. Suppose the first rule

$$(1) \text{ reg}_0 \leftarrow \text{const}_a \quad \{ \text{MOV } \#a, R0 \}$$

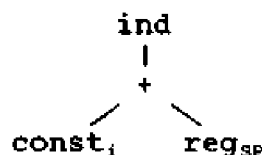
is applied to load the constant a into register $R0$. The label of the leftmost leaf then changes from const_a to reg_0 and the instruction $\text{MOV } \#a, R0$ is generated. The seventh rule

$$(7) \text{ reg}_0 \leftarrow \begin{array}{c} + \\ \swarrow \quad \searrow \\ \text{reg}_0 \quad \text{reg}_{SP} \end{array} \quad \{ \text{ADD } SP, R0 \}$$

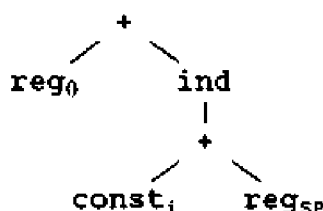
now matches the leftmost subtree with root labeled $+$. Using this rule, we rewrite this subtree as a single node labeled reg_0 and generate the instruction $\text{ADD } SP, R0$. Now the tree looks like



At this point, we could apply rule (5) to reduce the subtree

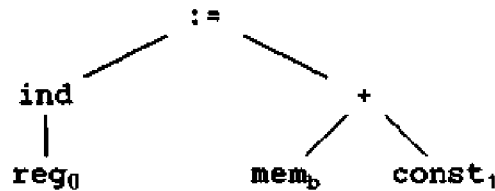


to a single node labeled reg_1 . However, we can also use rule (6) to reduce the larger subtree



into a single node labeled reg_0 and generate the instruction $\text{ADD } i(SP), R0$.

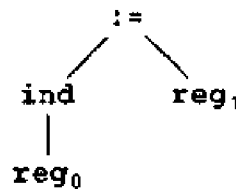
Assuming it is more efficient to use a single instruction to compute the larger subtree rather than the smaller one, we choose the latter reduction to get



In the right subtree, rule (2) applies to the leaf mem_b . This rule generates an instruction to load b into register 1, say. Now, using rule (8) we can match the subtree



and generate the increment instruction `INC R1`. At this point, the input tree has been reduced to



This remaining tree is matched by rule (4), which reduces the tree to a single node and generates the instruction `MOV R1, *R0`.

In the process of reducing the tree to a single node, we generate the following code sequence:

```

MOV #a,R0
ADD SP,R0
ADD 1(SP),R0
MOV b,R1
INC R1
MOV R1,*R0
  
```

□

Several aspects of this tree-reduction process need further explication. We have not specified how the tree-pattern matching is to be done. Nor have we specified an order in which the templates are matched or what to do if more than one template matches at a given time. Also, notice that if no template matches, then the code-generation process blocks. On the other extreme, it may be possible for a single node to be rewritten indefinitely, generating an infinite sequence of register move instructions or an infinite sequence of loads and stores.

One way to do the tree-pattern matching efficiently is to extend the multiple-keyword pattern-matching algorithm in Exercise 3.32 into a top-down

tree-pattern matching algorithm. Each template can be represented by a set of strings, namely, by the set of paths from the root to the leaves. From these collections of strings, we can construct a tree-pattern matcher as in Exercise 3.32.

The ordering and multiple-match problems can be resolved by using tree-pattern matching in conjunction with the dynamic programming algorithm of the previous section. A tree-translation scheme can be augmented with cost information by associating with each tree-rewriting rule the cost of the sequence of machine instructions generated if that rule is applied.

In practice, the tree-rewriting process can be implemented by running the tree-pattern matcher during a depth-first traversal of the input tree and performing the reductions as the nodes are visited for the last time. If we run the dynamic programming algorithm concurrently, we can select an optimal sequence of matches using the cost information associated with each rule. We may need to defer deciding upon a match until the cost of all alternatives is known. Using this approach, a small, efficient code generator can be constructed quickly from a tree-rewriting scheme. Moreover, the dynamic programming algorithm frees the code-generator designer from having to resolve conflicting matches or decide upon an order for the evaluation.

Pattern Matching by Parsing

Another approach is to use an LR parser to do the pattern matching. The input tree can be treated as a string by using its prefix representation. For example, the prefix representation for the tree in Fig. 9.30 is

$:= \text{ind} + + \text{const}_a \text{reg}_{sp} \text{ind} + \text{const}_i \text{reg}_{sp} + \text{mem}_b \text{const}_1$

The tree-translation scheme can be converted into a syntax-directed translation scheme by replacing the tree-rewriting rules with the productions of a context-free grammar in which the right sides are prefix representations of the instruction templates.

Example 9.18. The syntax-directed translation scheme in Fig. 9.32 is based on the tree translation scheme in Fig. 9.31. $\square$

From the productions of the translation scheme we build an LR parser using one of the LR-parser construction techniques of Chapter 4. The target code is generated by emitting the machine instruction corresponding to each reduction.

A code-generation grammar is usually highly ambiguous, and some care needs to be given to how the parsing-action conflicts are resolved when the parser is constructed. In the absence of cost information, a general rule is to favor larger reductions over smaller ones. This means that in a reduce-reduce conflict, the longer reduction is favored; in a shift-reduce conflict, the shift move is chosen. This "maximal munch" approach causes a larger number of operations to be performed with a single machine instruction.

There are several aspects to using LR parsing in code generation. First, the

(1)	$\text{reg}_i \rightarrow \text{const}_c$	{ MOV #c, Ri }
(2)	$\text{reg}_i \rightarrow \text{mem}_a$	{ MOV a, Ri }
(3)	$\text{mem} \rightarrow := \text{mem}_a \text{ reg}_i$	{ MOV Ri, a }
(4)	$\text{mem} \rightarrow := \text{ind reg}_i \text{ reg}_j$	{ MOV Rj, *Ri }
(5)	$\text{reg}_i \rightarrow \text{ind} + \text{const}_c \text{ reg}_j$	{ MOV c(Rj), Ri }
(6)	$\text{reg}_i \rightarrow + \text{reg}_i \text{ ind} + \text{const}_c \text{ reg}_j$	{ ADD c(Rj), Ri }
(7)	$\text{reg}_i \rightarrow + \text{reg}_i \text{ reg}_j$	{ ADD Rj, Ri }
(8)	$\text{reg}_i \rightarrow + \text{reg}_i \text{ const}_1$	{ INC Ri }

Fig. 9.32. Syntax-directed translation scheme constructed from Fig. 9.31.

parsing method is efficient and well understood, so reliable and efficient code generators can be produced using the algorithms described in Chapter 4. Second, it is relatively easy to retarget the resulting code generator; a code selector for a new machine can be constructed by writing a grammar to describe the instructions of the new machine. Third, the quality of the code generated can be made efficient by adding special-case productions to take advantage of machine idioms.

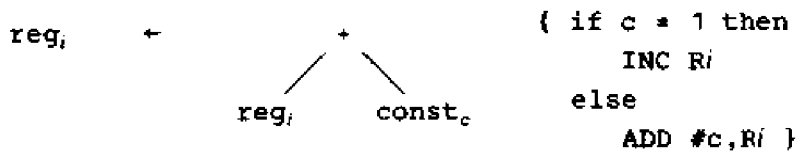
However, there are some difficulties as well. A left-to-right order of evaluation is fixed by the parsing method. Also, for some machines with large numbers of addressing modes, the machine-description grammar and resulting parser can become inordinately large. As a consequence, specialized techniques are necessary to encode and process the machine-description grammars. We must also be careful that the resulting parser does not block (has no next move) while parsing on an expression tree, because the grammar does not handle some operator patterns or the parser has made the wrong resolution of some parsing-action conflict. We must also make sure the parser does not get into an infinite loop of reductions of productions with single symbols on the right side. The looping problem can be solved using a state-splitting technique at the time the parser tables are generated (see Glanville [1977]).

Routines for Semantic Checking

The leaves in the input tree are type attributes with subscripts, where a subscript associates a value with an attribute. In a code-generation translation scheme, the same attributes appear, but often with restrictions on what values the subscripts can have. For example, a machine instruction may require that an attribute value fall in a certain range or that the values of two attributes be related.

These restrictions on attribute values can be specified as predicates that are invoked before a reduction is made. In fact, the general use of semantic actions and predicates can provide greater flexibility and ease of description than a purely grammatical specification of a code generator. Generic templates can be used to represent classes of instructions and the semantic actions can then be used to pick instructions for specific cases. For example, two

forms of the addition instruction can be represented with one template:



Parsing-action conflicts can be resolved by disambiguating predicates that can allow different selection strategies to be used in different contexts. A smaller description of a target machine is possible because certain aspects of the machine architecture, such as addressing modes, can be factored into the attributes. The complication in this approach is that it may become difficult to verify the accuracy of the attribute grammar as a faithful description of the target machine, although this problem is shared to some degree by all code generators.

EXERCISES

- 9.1** Generate code for the following C statements for the target machine of Section 9.2 assuming all variables are static. Assume three registers are available.
- $x = 1$
 - $x = y$
 - $x = x + 1$
 - $x = a + b * c$
 - $x = a / (b + c) - d * (e + f)$
- 9.2** Repeat Exercise 9.1 assuming all variables are automatic (allocated on the stack).
- 9.3** Generate code for the following C statements for the target machine of Section 9.2 assuming all variables are static. Assume three registers are available.
- $x = a[i] + 1$
 - $a[i] = b[c[i]]$
 - $a[i][j] = b[i][k] * c[k][j]$
 - $a[i] = a[i] + b[j]$
 - $a[i] += b[j]$
- 9.4** Do Exercise 9.1 using
- the algorithm in Section 9.6
 - the procedure *encode*
 - the dynamic programming algorithm of Section 9.11
- 9.5** Generate code for the following C statements

- a) $x = f(a) + f(a) + f(a)$
- b) $x = f(a)/g(b,c)$
- c) $x = f(f(a))$
- d) $x = ++f(a)$
- e) $*p++ = *q++$

9.6 Generate code for the following C program

```
main()
{
    int i;
    int a[10];
    while (i <= 10)
        a[i] = 0;
}
```

- 9.7** Suppose that for the loop of Fig. 9.13 we choose to allocate three registers to hold a , b , and c . Generate code for the blocks of this loop. Compare the cost of your code with that in Fig. 9.14.
- 9.8** Construct the register-interference graph for the program in Fig. 9.13.
- 9.9** Suppose that for simplicity we automatically store all registers on the stack (or in memory if a stack is not used) before each procedure call and restore them after the return. How does this affect the formula (9.4) used to evaluate the utility of allocating a register to a given variable in a loop?
- 9.10** Modify the function *getreg* of Section 9.6 to return register-pairs when needed.
- 9.11** Give an example of a dag for which the heuristic for ordering the nodes of a dag given in Fig. 9.21 does not give the best ordering.
- *9.12** Generate optimal code for the following assignment statements:
- a) $x := a + b * c$
 - b) $x := (a * -b) + (c - (d + e))$
 - c) $x := (a/b - c)/d$
 - d) $x := a + (b + c/d * e)/(f * g - h * i)$
 - e) $a[i,j] := b[i,j] - c[a[k,l]] * d[i+j]$
- 9.13** Generate code for the following Pascal program:

```
program forloop(input, output);
var i, initial, final: integer;
begin
    read(initial, final);
    for i:= initial to final do
        writeln(i)
    end.
```

9.14 Construct the dag for the following basic block.

```

d := b * c
e := a + b
b := b * c
a := e - d

```

9.15 What are the legal evaluation orders and names for the values at the nodes for the dag of Exercise 9.14

- a) assuming a, b and c are live at the end of the basic block?
- b) assuming only a is live at the end?

9.16 In Exercise 9.15(b), if we are going to generate code for a machine with only one register, which evaluation order is best? Why?

9.17 We can modify the dag construction algorithm to take into account assignments to arrays and through pointers. When any element of an array is assigned, we assume a new value for that array is created. This new value is represented by a node whose children are the old value of the array, the value of the index into the array, and the value assigned. When an assignment through a pointer occurs, we assume we have created a new value for every variable that the pointer might have pointed to; the children of the node for each new value are the value of the pointer and the old value of the variable that might have been assigned. Using these assumptions, construct the dag for the following basic block:

```

a[i] := b
*p := c
d := a[j]
e := *p
*p := a[i]

```

Assume that (a) p can point anywhere, (b) p points to only b or d. Do not forget to show the implied order constraints.

9.18 If a pointer or array expression such as a[i] or *p is assigned and then used without the possibility of its value having changed in the interim, we can recognize and take advantage of the situation to simplify the dag. For example, in the code of Exercise 9.17, since p is not assigned between the second and fourth statements, the statement e := *p can be replaced by e := c, since we are sure that whatever p points to has the same value as c, even though we don't know what p points to. Revise the dag construction algorithm to take advantage of such inferences. Apply your algorithm to the code of Exercise 9.17.

- **9.19** Devise an algorithm to generate optimal code for a sequence of three-address statements of the form $a := b + c$ on the n -register machine of Example 9.14. The statements must be executed in the order given. What is the time complexity of your algorithm?

BIBLIOGRAPHIC NOTES

The reader interested in overviews of code-generation research should consult Waite [1976a,b], Aho and Sethi [1977], Graham [1980 and 1984], Ganapathi, Fischer, and Hennessy [1982], Lunell [1983], and Henry [1984]. Code generation for Bliss is discussed by Wulf et al. [1975], for Pascal by Ammann [1977], and for PL.8 by Ausländer and Hopkins [1982].

Program-usage statistics are useful for compiler design. Knuth [1971b] did an empirical study of Fortran programs. Elshoff [1976] provides some statistics on PL/I usage, and Shimasaki et al. [1980] and Carter [1982] analyze Pascal programs. The performance of several compilers on various computer instruction sets is discussed by Lunde [1977], Shustek [1978], and Ditzel and McLellan [1982].

Many of the heuristics for code generation proposed in this chapter have been used in various compilers. Freiburghouse [1974] discusses usage counts as an aid to generating good code for basic blocks. The strategy used in *getreg* of creating a free register by throwing out of a register the variable whose value will be unused for the longest time was shown optimal in a page-swapping context by Belady [1966]. Our strategy of allocating a fixed number of registers to hold variables for the duration of a loop was mentioned by Marill [1962] and used in the implementation of Fortran H by Lowry and Medlock [1969].

Horwitz et al. [1966] give an algorithm for optimizing the use of index registers in Fortran. Graph coloring as a register-allocation technique was proposed by J. Cocke, Ershov [1971], and Schwartz [1973]. The treatment of graph coloring in Section 9.7 follows Chaitin et al. [1981] and Chaitin [1982]. Chow and Hennessy [1984] describe a priority-based graph coloring algorithm for register allocation. Other approaches to register allocation are discussed by Kennedy [1972], Johnsson [1975], Harrison [1975], Beatty [1974], and Leverett [1982].

The labeling algorithm for trees in Section 9.10 is reminiscent of an algorithm for naming rivers: the confluence of a major river and a minor tributary continues to use the name of the major river; the confluence of two equal-sized rivers is given a new name. The labeling algorithm originally appeared in Ershov [1958]. Code-generation algorithms using this method have been proposed by Anderson [1964], Nievergelt [1965], Nakata [1967], Redziejewski [1969], and Beatty [1972]. Sethi and Ullman [1970] used the labeling method in an algorithm that they were able to prove generated optimal code for expression trees in a wide variety of situations. The procedure *gencode* in Section 9.10 is a modification of Sethi and Ullman's algorithm, due to

Stockhausen [1973]. Bruno and Lassagne [1975] and Coffman and Sethi [1983] give optimal code-generation algorithms for expression trees if the target machine has registers that must be used as a stack.

Aho and Johnson [1976] devised the dynamic programming algorithm described in Section 9.11. This algorithm was used as the basis of the code generator in S.C. Johnson's portable C compiler, PCC2, and was also used by Ripken [1977] in a compiler for the IBM 370 machine. Knuth [1977] generalized the dynamic programming algorithm to machines with asymmetric register classes, such as the IBM 7090 and the CDC 6600. In developing the generalization, Knuth viewed code generation as a parsing problem for context-free grammars.

Floyd [1961] gives an algorithm to handle common subexpressions in arithmetic expressions. The partition of dags into trees and the use of a procedure like *gencode* on the trees separately is from Waite [1976a]. Sethi [1975] and Bruno and Sethi [1976] show that the optimal code-generation problem for dags is NP-complete. Aho, Johnson, and Ullman [1977a] show that the problem remains NP-complete even with single-register and infinite-register machines. Aho, Hopcroft, and Ullman [1974] and Garey and Johnson [1979] discuss the significance of what it means for a problem to be NP-complete.

Transformations on basic blocks have been studied by Aho and Ullman [1972a] and by Downey and Sethi [1978]. Peephole optimization is discussed by McKeeman [1965], Fraser [1979], Davidson and Fraser [1980 and 1984a,b], Lamb [1981], and Giegerich [1983]. Tanenbaum, van Staveren, and Stevenson [1982] advocate using peephole optimization on intermediate code as well.

Code generation has been treated as a tree-rewriting process by Wasilew [1971], Weingart [1973], Johnson [1978], and Cattell [1980]. The tree-rewriting example in Section 9.12 is derived from Henry [1984]. Aho and Ganapathi [1985] proposed the combination of efficient tree-pattern matching with dynamic programming mentioned in the same section. Tjiang [1986] has implemented a code-generation language called Twig based on the tree-translation schemes in Section 9.12. Kron [1975], Huet and Levy [1979], and Hoffman and O'Donnell [1982] describe general algorithms for tree-pattern matching.

The Graham-Glanville approach to code generation by using an LR parser for instruction selection is described and evaluated in Glanville [1977], Glanville and Graham [1978], Graham [1980 and 1984], Henry [1984], and Aigrain et al. [1984]. Ganapathi [1980] and Ganapathi and Fischer [1982] have used attribute grammars to specify and implement code generators.

Other techniques for automating the construction of code generators have been proposed by Fraser [1977], Cattell [1980], and Leverett et al. [1980]. Compiler portability is also discussed by Richards [1971 and 1977]. Szymanski [1978] and Leverett and Szymanski [1980] describe techniques for chaining span-dependent jump instructions. Yannakakis [1985] has a polynomial-time algorithm for Exercise 9.19.

Code Optimization

Ideally, compilers should produce target code that is as good as can be written by hand. The reality is that this goal is achieved only in limited cases, and with difficulty. However, the code produced by straightforward compiling algorithms can often be made to run faster or take less space, or both. This improvement is achieved by program transformations that are traditionally called *optimizations*, although the term “optimization” is a misnomer because there is rarely a guarantee that the resulting code is the best possible. Compilers that apply code-improving transformations are called *optimizing compilers*.

The emphasis in this chapter is on machine-independent optimizations, program transformations that improve the target code without taking into consideration any properties of the target machine. Machine-dependent optimizations, such as register allocation and utilization of special machine-instruction sequences (machine idioms) were discussed in Chapter 9.

The most payoff for the least effort is obtained if we can identify the frequently executed parts of a program and then make these parts as efficient as possible. There is a popular saying that most programs spend ninety per cent of their execution time in ten per cent of the code. While the actual percentages may vary, it is often the case that a small fraction of a program accounts for most of the running time. Profiling the run-time execution of a program on representative input data accurately identifies the heavily traveled regions of a program. Unfortunately, a compiler does not have the benefit of sample input data, so it must make its best guess as to where the program hot spots are.

In practice, the program’s inner loops are good candidates for improvement. In a language that emphasizes control constructs like while and for statements, the loops may be evident from the syntax of the program; in general, a process called control-flow analysis identifies loops in the flow graph of a program.

This chapter is a cornucopia of useful optimizing transformations and techniques for implementing them. The best technique for deciding what transformations are worthwhile to put into a compiler is to collect statistics about the source programs and evaluate the benefit of a given set of optimizations on a

representative sample of real source programs. Chapter 12 describes transformations that have proven useful in optimizing compilers for several different languages.

One of the themes of this chapter is data-flow analysis, a process of collecting information about the way variables are used in a program. The information collected at various points in a program can be related using simple set equations. We present several algorithms for collecting information using data-flow analysis and for effectively using this information in optimization. We also consider the impact of language constructs such as procedures and pointers on optimization.

The last four sections of this chapter deal with more advanced material. They cover some graph-theoretic ideas relevant to control-flow analysis and apply these ideas to data-flow analysis. The chapter concludes with a discussion of general-purpose tools for data-flow analysis and techniques for debugging optimized code. The emphasis throughout this chapter is on optimizing techniques that apply to languages in general. Some compilers that use these ideas are reviewed in Chapter 12.

10.1 INTRODUCTION

To create an efficient target language program, a programmer needs more than an optimizing compiler. In this section, we review the options available to a programmer and a compiler for creating efficient target programs. We mention the types of code-improving transformations that a programmer and a compiler writer can be expected to use to improve the performance of a program. We also consider the representation of programs on which transformations will be applied.

Criteria for Code-Improving Transformations

Simply stated, the best program transformations are those that yield the most benefit for the least effort. The transformations provided by an optimizing compiler should have several properties.

First, a transformation must preserve the meaning of programs. That is, an "optimization" must not change the output produced by a program for a given input, or cause an error, such as a division by zero, that was not present in the original version of the source program. The influence of this criterion pervades this chapter; at all times we take the "safe" approach of missing an opportunity to apply a transformation rather than risk changing what the program does.

Second, a transformation must, on the average, speed up programs by a measurable amount. Sometimes we are interested in reducing the space taken by the compiled code, although the size of code has less importance than it once had. Of course, not every transformation succeeds in improving every program, and occasionally an "optimization" may slow down a program slightly, as long as on the average it improves things.

Third, a transformation must be worth the effort. It does not make sense for a compiler writer to expend the intellectual effort to implement a code-improving transformation and to have the compiler expend the additional time compiling source programs if this effort is not repaid when the target programs are executed. Certain local or “peephole” transformations of the kind discussed in Section 9.9 are simple enough and beneficial enough to be included in any compiler.

Some transformations can only be applied after detailed, often time-consuming, analysis of the source program, so there is little point in applying them to programs that will be run only a few times. For example, a fast, nonoptimizing, compiler is likely to be more helpful during debugging or for “student jobs” that will be run successfully a few times and thrown away. Only when the program in question takes up a significant fraction of the machine’s cycles does improved code quality justify the time spent running an optimizing compiler on the program.

Getting Better Performance

Dramatic improvements in the running time of a program — such as cutting the running time from a few hours to a few seconds — are usually obtained by improving the program at all levels, from the source level to the target level, as suggested by Fig. 10.1. At each level, the available options fall between the two extremes of finding a better algorithm and of implementing a given algorithm so that fewer operations are performed.

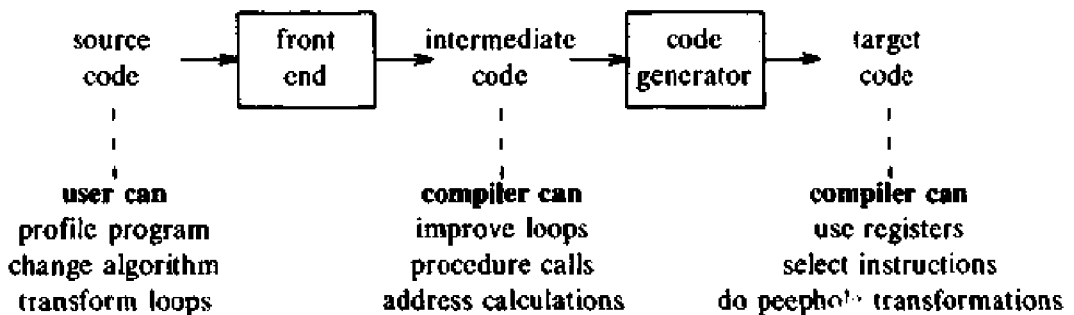


Fig. 10.1. Places for potential improvements by the user and the compiler.

Algorithmic transformations occasionally produce spectacular improvements in running time. For example, Bentley [1982] relates that the running time of a program for sorting N elements dropped from $2.02N^2$ microseconds to $12N\log_2 N$ microseconds when a carefully coded “insertion sort” was replaced by “quicksort.”¹ For $N = 100$ the replacement speeds up the program by a

¹ See Aho, Hopcroft, and Ullman [1983] for a discussion of these sorting algorithms and their speeds.

factor of 2.5. For $N = 100,000$ the improvement is far more dramatic: the replacement speeds up the program by a factor of more than a thousand.

Unfortunately, no compiler can find the best algorithm for a given program. Sometimes, however, a compiler can replace a sequence of operations by an algebraically equivalent sequence, and thereby reduce the running time of a program significantly. Such savings are more common when algebraic transformations are applied to programs in very-high level languages, e.g., query languages for databases (see Ullman [1982]).

In this section and the next, a sorting program called quicksort will be used to illustrate the effect of various code-improving transformations. The C program in Fig. 10.2 is derived from Sedgewick [1978], where hand-optimization of such a program is discussed. We shall not discuss the algorithmic aspects of this program here — in fact, $a[0]$ must contain the smallest and $a[\text{max}]$ the largest element to be sorted for the program to work.

```
void quicksort(m,n)
int m,n;
{
    int i,j;
    int v,x;
    if ( n <= m ) return;
    /* fragment begins here */
    i = m-1; j = n; v = a[n];
    while(1) {
        do i = i+1; while ( a[i] < v );
        do j = j-1; while ( a[j] > v );
        if ( i >= j ) break;
        x = a[i]; a[i] = a[j]; a[j] = x;
    }
    x = a[i]; a[i] = a[n]; a[n] = x;
    /* fragment ends here */
    quicksort(m,j); quicksort(i+1,n);
}
```

Fig. 10.2. C code for quicksort.

It may not be possible to perform certain code-improving transformations at the level of the source language. For example, in a language such as Pascal or Fortran, a programmer can only refer to array elements in the usual way, e.g., as $b[i,j]$. At the level of the intermediate language, however, new opportunities for code improvement may be exposed. Three-address code, for example, provides many opportunities for improving address calculations, especially in loops. Consider the three-address code for determining the value of $a[i]$, assuming that each array element takes four bytes:

```
t1 := 4*i;  t2 := a[t1]
```

Naive intermediate code will recalculate $4*i$ every time $a[i]$ appears in the source program, and the programmer has no control over the redundant address calculations, because they are implicit in the implementation of the language, rather than being explicit in the code written by the user. In these situations, it behooves the compiler to clean them up. In a language like C, however, this transformation can be done at the source level by the programmer, since references to array elements can be systematically rewritten using pointers to make them more efficient. This rewriting is similar to transformations that optimizing compilers for Fortran traditionally apply.

At the level of the target machine, it is the compiler's responsibility to make good use of the machine's resources. For example, keeping the most heavily used variables in registers can cut running time significantly, often by as much as a half. Again, C allows a programmer to advise the compiler that certain variables be held in registers, but most languages do not. Similarly, the compiler can speed up programs significantly by choosing instructions that take advantage of the addressing modes of the machine to do in one instruction what naively we might expect to require two or three, as we discussed in Chapter 9.

Even if it is possible for the programmer to improve the code, it may be more convenient to have the compiler make some of the improvements. If a compiler can be relied upon to generate efficient code, then the user can concentrate on writing clear code.

An Organization for an Optimizing Compiler

As we have mentioned, there are often several levels at which a program can be improved. Since the techniques needed to analyze and transform a program do not change significantly with the level, this chapter concentrates on the transformation of intermediate code using the organization shown in Fig. 10.3. The code-improvement phase consists of control-flow and data-flow analysis followed by the application of transformations. The code generator, discussed in Chapter 9, produces the target program from the transformed intermediate code.

For convenience of presentation, we assume that the intermediate code consists of three-address statements. Intermediate code, of the sort produced by the techniques in Chapter 8, for a portion of the program in Fig. 10.2 is shown in Fig. 10.4. With other intermediate representations, the temporary variables $t_1, t_2, \dots, t_{15}$ in Fig. 10.4 need not appear explicitly, as discussed in Chapter 8.

The organization in Fig. 10.3 has the following advantages:

1. The operations needed to implement high-level constructs are made explicit in the intermediate code, so it is possible to optimize them. For example, the address calculations for $a[i]$ are explicit in Fig. 10.4, so the recomputation of expressions like $4*i$ can be eliminated as discussed in the next section.

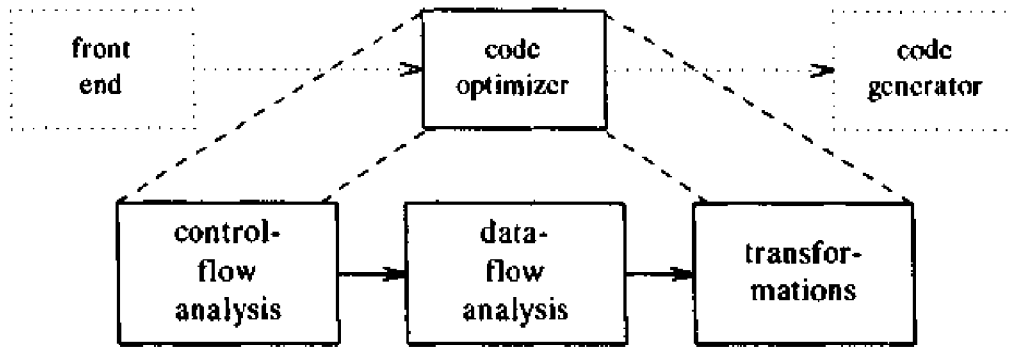


Fig. 10.3. Organization of the code optimizer.

(1) $i := m - 1$	(16) $t_7 := 4 * i$
(2) $j := n$	(17) $t_8 := 4 * j$
(3) $t_1 := 4 * n$	(18) $t_9 := a[t_8]$
(4) $v := a[t_1]$	(19) $a[t_7] := t_9$
(5) $i := i + 1$	(20) $t_{10} := 4 * j$
(6) $t_2 := 4 * i$	(21) $a[t_{10}] := x$
(7) $t_3 := a[t_2]$	(22) goto (5)
(8) if $t_3 < v$ goto (5)	(23) $t_{11} := 4 * i$
(9) $j := j - 1$	(24) $x := a[t_{11}]$
(10) $t_4 := 4 * j$	(25) $t_{12} := 4 * i$
(11) $t_5 := a[t_4]$	(26) $t_{13} := 4 * n$
(12) if $t_5 > v$ goto (9)	(27) $t_{14} := a[t_{13}]$
(13) if $i \geq j$ goto (23)	(28) $a[t_{12}] := t_{14}$
(14) $t_6 := 4 * i$	(29) $t_{15} := 4 * n$
(15) $x := a[t_6]$	(30) $a[t_{15}] := x$

Fig. 10.4. Three-address code for fragment in Fig. 10.2.

- The intermediate code can be (relatively) independent of the target machine, so the optimizer does not have to change much if the code generator is replaced by one for a different machine. The intermediate code in Fig. 10.4 assumes that each element of the array a takes four bytes. Some intermediate codes, e.g., P-code for Pascal, leave it to the code generator to fill in the size of array elements, so the intermediate code is independent of the size of a machine word. We could have done the same in our intermediate code if we replaced 4 by a symbolic constant.

In the code optimizer, programs are represented by flow graphs, in which edges indicate the flow of control and nodes represent basic blocks, as discussed in Section 9.4. Unless otherwise specified, a program means a single procedure. In Section 10.8, we discuss interprocedural optimization.

Example 10.1. Figure 10.5 contains the flow graph for the program in Fig. 10.4. B_1 is the initial node. All conditional and unconditional jumps to statements in Fig. 10.4 have been replaced in Fig. 10.5 by jumps to the block of which the statements are leaders.

In Fig. 10.5, there are three loops. B_2 and B_3 are loops by themselves. Blocks B_2 , B_3 , B_4 , and B_5 together form a loop, with entry B_2 . $\square$

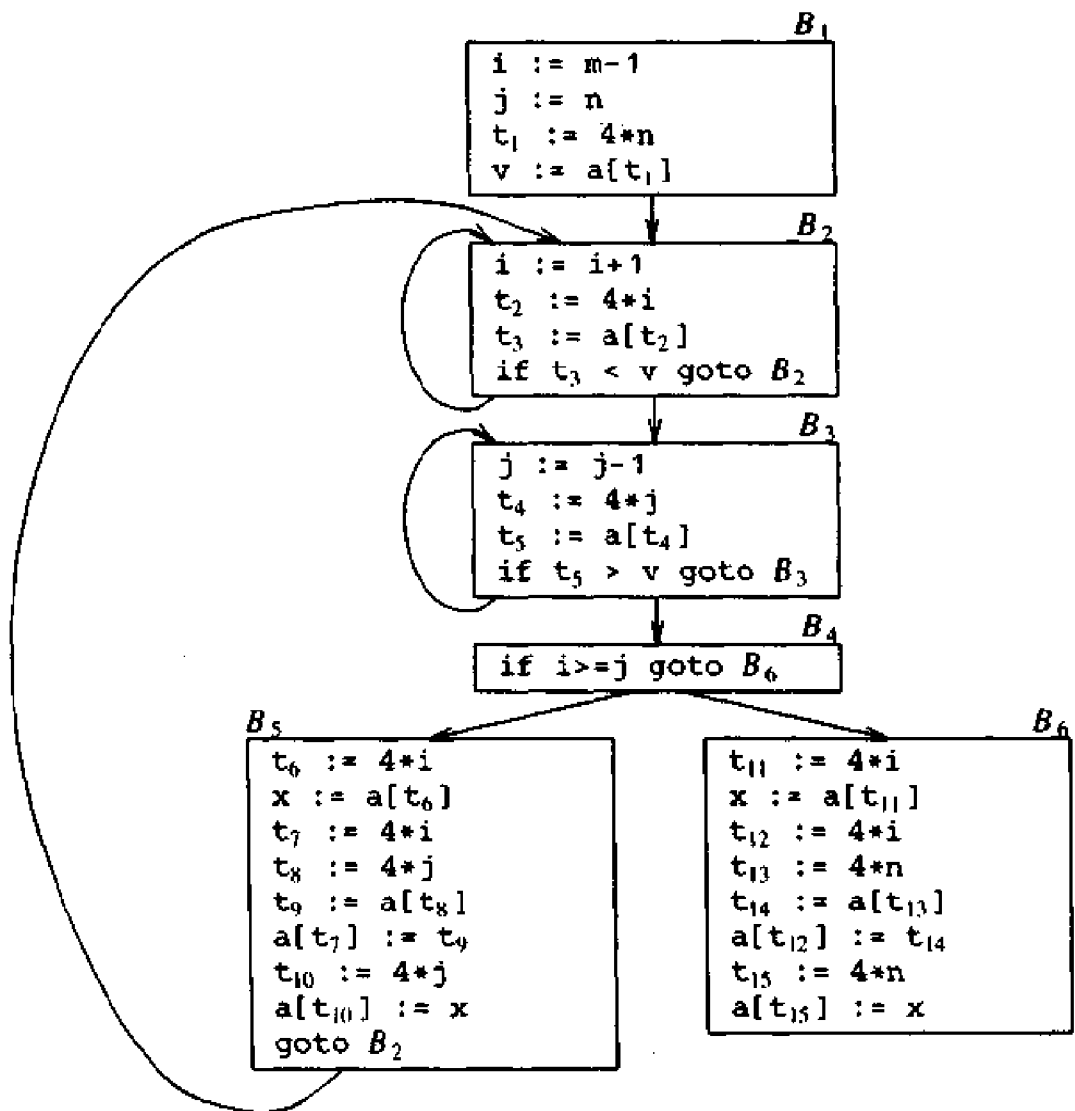


Fig. 10.5. Flow graph.

10.2 THE PRINCIPAL SOURCES OF OPTIMIZATION

In this section, we introduce some of the most useful code-improving transformations. Techniques for implementing these transformations are presented in subsequent sections. A transformation of a program is called *local* if it can be performed by looking only at the statements in a basic block; otherwise, it is called *global*. Many transformations can be performed at both the local and global levels. Local transformations are usually performed first.

Function-Preserving Transformations

There are a number of ways in which a compiler can improve a program without changing the function it computes. Common subexpression elimination, copy propagation, dead-code elimination, and constant folding are common examples of such function-preserving transformations. Section 9.8 on the dag representation of basic blocks showed how local common subexpressions could be removed as the dag for the basic block was constructed. The other transformations come up primarily when global optimizations are performed, and we shall discuss each in turn.

Frequently, a program will include several calculations of the same value, such as an offset in an array. As mentioned in Section 10.1, some of these duplicate calculations cannot be avoided by the programmer because they lie below the level of detail accessible within the source language. For example, block B_5 shown in Fig. 10.6(a) recalculates $4*i$ and $4*j$.

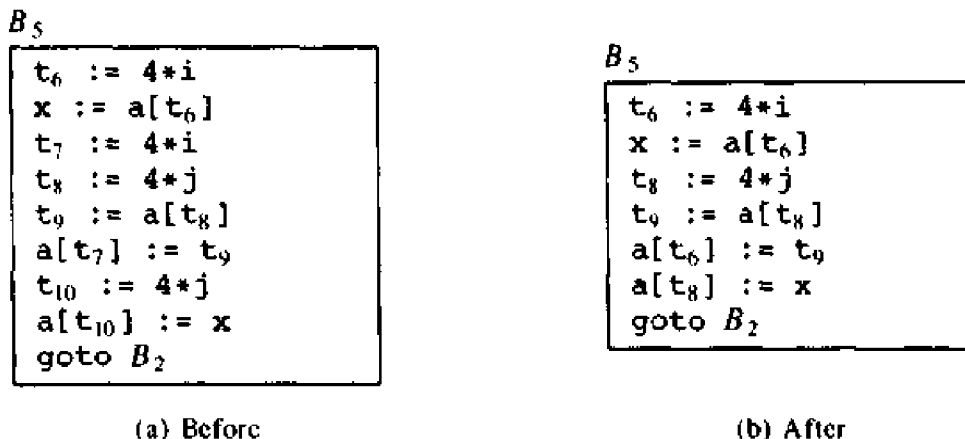


Fig. 10.6. Local common subexpression elimination.

Common Subexpressions

An occurrence of an expression E is called a *common subexpression* if E was previously computed, and the values of variables in E have not changed since the previous computation. We can avoid recomputing the expression if we can use the previously computed value. For example, the assignments to t_7 and